

APPENDIX B. EXAMPLES

MIKROELEKTRONIKA

The purpose of this chapter is to provide basic information about microcontrollers that one needs to know in order to be able to use them successfully in practice. This chapter, therefore, does not contain any super interesting program or device schematic with amazing solutions. Instead, the following examples are more proof that program writing is neither a privilege nor a talent issue but the ability of simply putting puzzle pieces together using directives. Rest assured that design and development of devices mainly consists of the following method “test-correct-repeat”. Of course, the more you are in it the more complicated it becomes since the puzzle pieces are put together by both children and first-class architects...

BASIC CONNECTING

As seen in the figure below, in order to enable the microcontroller to operate properly it is necessary to provide:

- Power Supply;
- Reset Signal; and
- Clock Signal.

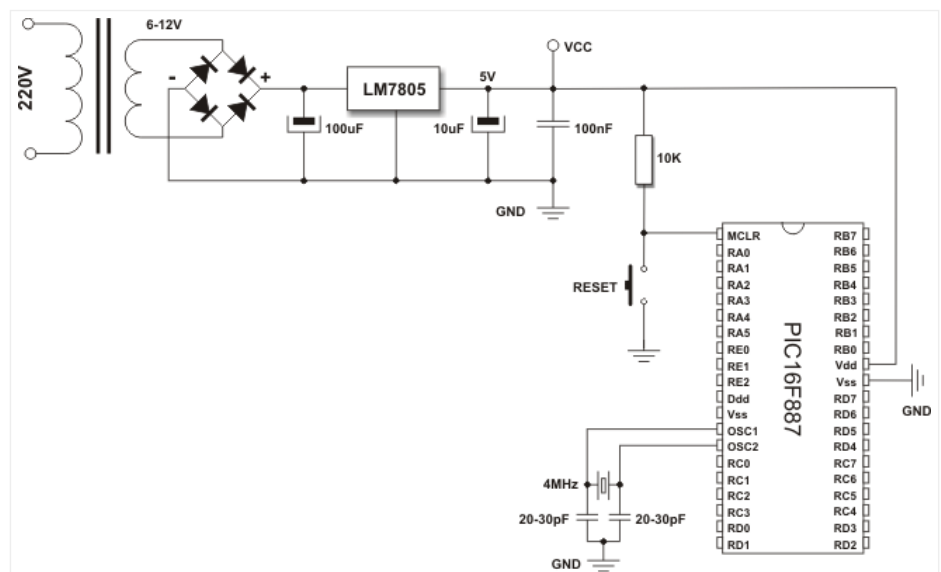
Clearly, it is about simple circuits, but it does not have to always be like that. If the target device is used for controlling expensive machines or maintaining vital functions, everything gets more and more complicated! However, this solution is sufficient for the time being...

POWER SUPPLY

Even though the PIC16F887 can operate at different supply voltages, why to test “Murphy’s law”?! A 5VDC power supply is shown above. The circuit, uses a cheap integrated three-terminal positive regulator, LM7805, provides high-quality voltage stability and quite enough current to enable microcontroller and peripheral electronics to operate normally (enough in this case means 1Amp).

RESET SIGNAL

In order that the microcontroller can operate properly, a logic one (VCC) must be applied on the reset pin it explains the connection pin-resistor 10K-VCC. The push-button connecting the reset pin MCLR to GND is not necessary. However, it is almost always provided because it enables the microcontroller safe return to normal

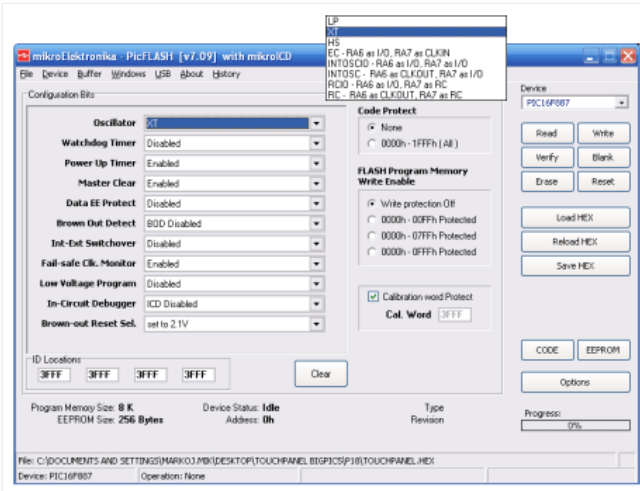


operating conditions if something goes wrong. By pushing this button, 0V is brought to the pin, the microcontroller is reset and program execution starts from the beginning. The 10K resistor is there to allow 0V to be applied to the MCLR pin, via the push-button, without shorting the 5VDC rail to ground.

CLOCK SIGNAL

Even though the microcontroller has a built in oscillator, it cannot operate without external components which stabilize its operation and determine its frequency (operating speed of the microcontroller). Depending on which elements are in use as well as their frequencies, the oscillator can be run in four different modes:

- LP – Low Power Crystal;
- XT – Crystal / Resonator;
- HS – High speed Crystal / Resonator; and
- RC – Resistor / Capacitor.



Quartz resonator

When a quartz crystal is used for frequency stabilization, the built in oscillator operates at a very precise frequency which is isolated from changes in temperature and voltage power supply as well. This frequency is normally labelled on the microcontroller package.

Apart from the crystal, capacitors C1 and C2 must be also connected as per the schematic below. Their capacitance is not of great importance, therefore, the values provided in the table should be considered as a

Why are these modes so important? Owing to the fact that it is almost impossible to make a stable oscillator which operates over a wide frequency range, the microcontroller must know which crystal is connected in order that it can adjust the operation of its internal electronics to it. This is why all programs used for chip loading contains an option for oscillator mode selection. See above figure.

recommendation rather than a strict rule.

Mode	Frequency	C1, C2
LP	32 KHz	33pF
	200 KHz	15pF
XT	200 KHz	47-68 pF
	1 MHz	15 pF
	4 MHz	15 pF
HS	4 MHz	15 pF
	8 MHz	15-33 pF
	20 MHz	15-33 pF

Ceramic resonator

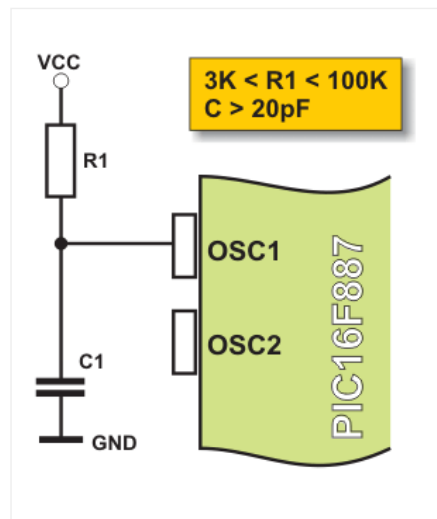
Ceramic resonator is cheaper, but very similar to quartz by its function and the way of operating. This is why the schematics illustrating their connection to the microcontroller are identical. However, the capacitor value is a bit different in this case due to different electric features. Refer to the table.

Mode	Frequency	C1, C2
XT	455 KHz	68-100 pF
	2 MHz	15-68 pF
	4 MHz	15-68 pF
HS	8 MHz	10-68 pF
	16 MHz	10-22 pF

These oscillators are used when it is not necessary to have extremely precise frequency.

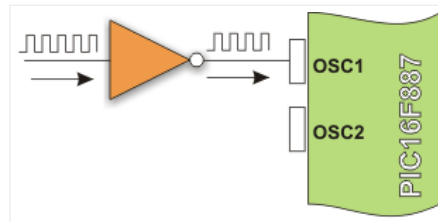
RC oscillator

If the operating frequency is not of importance then there is no need for additional expensive components for stabilization. Instead, a simple RC network, as shown in the figure below, will be enough. Since only the input of the local oscillator input is in use here, the clock signal with frequency $F_{osc}/4$ will appear on the OSC2 pin. Furthermore, that frequency becomes operating frequency of the microcontroller, i.e. the speed of instruction execution.



External oscillator

If it is required to synchronize the operation of several microcontrollers or if for some reason it is not possible to use any of the previous schematics, a clock signal may be generated by an external oscillator. Refer to the figure below.



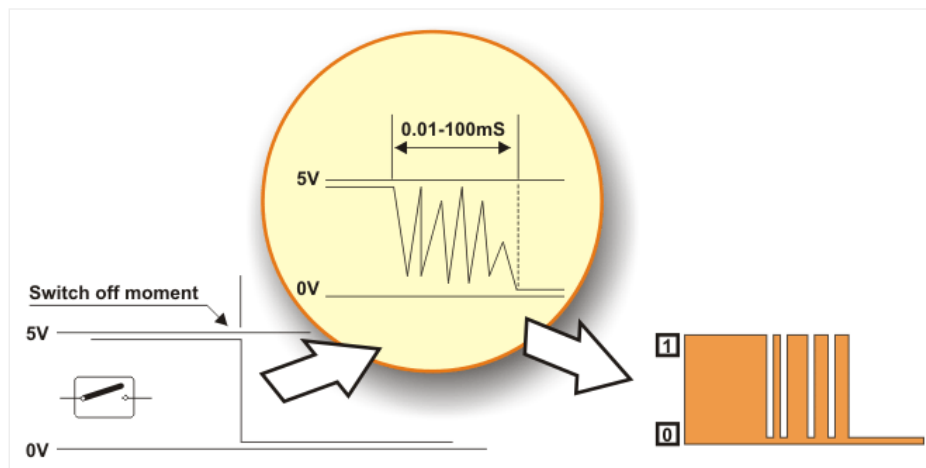
ADDITIONAL COMPONENTS

Regardless of the fact that the microcontroller is a product of modern technology, it is of no use without being connected to additional components. Simply, the appearance of voltage on the microcontroller pins mean nothing if not used for performing certain operations (turn something on/off, shift, display etc.).

This section intentionally covers only the most commonly used additional components in practice such as resistors, transistors, LED diodes, LED displays, LCD displays and RS232 communication circuits.

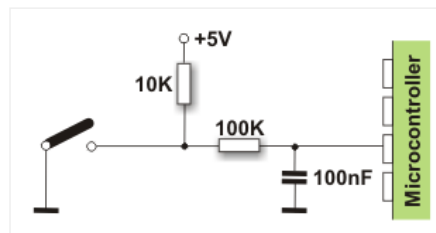
SWITCHES AND PUSH-BUTTONS

There is nothing simpler than switches and push-buttons! This is definitely the simplest way of detecting the appearance of a voltage on the microcontroller input pin and there is no need for additional explanation of how these components operate. Nevertheless, it is not so simple in practice... Then, what is it all about?

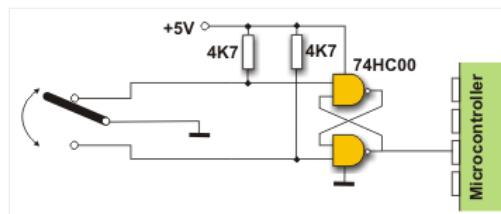


It is about contact bounce- a common problem with mechanical switches. When the contacts strike together, their momentum and elasticity act together to cause bounce. The result is a rapidly pulsed electrical current instead of a clean transition from zero to full current. Generally, it mostly occurs due to vibrations, slight rough spots and dirt between contacts. This effect is usually unnoticeable when using these components in everyday life because the bounce happens too quickly to affect most equipment, but causes problems in some analogue and logic circuits that respond fast enough to misinterpret the on-off pulses as a data stream. Anyway, the whole process does not last long (a few micro- or milliseconds), but long enough to be registered by the microcontroller. When using only a push-button as a pulse counter, errors occurs in almost 100% of cases!

This problem may be easily solved by connecting a simple RC circuit to suppress quick voltage changes. Since the bounce period is not defined, the values of components are not precisely determined. In most cases, it is recommended to use the values as shown in figure below.



If complete stability is needed then radical measures should be taken! The output of the circuit, shown in figure below (RS flip-flop), will change its logic state only after detecting the first pulse triggered by contact bounce. This solution is more expensive (SPDT switch), but the problem is definitely solved!

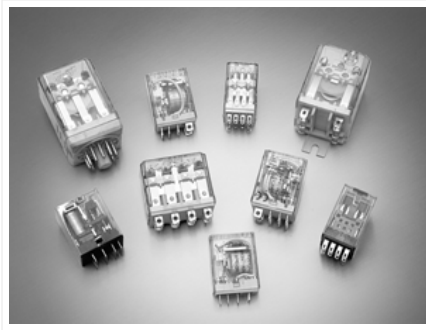


In addition to these hardware solutions, there is also a simple software solution. When a program tests the state of an input pin and detects a change, the check should be done one more time after a certain delay. If the program confirms the change, it means that a switch/push-button has changed its position. The advantages of such solution are obvious: it is free of charge, effects of noises are eliminated and it can be applied to the poorer quality contacts as well.

RELAY

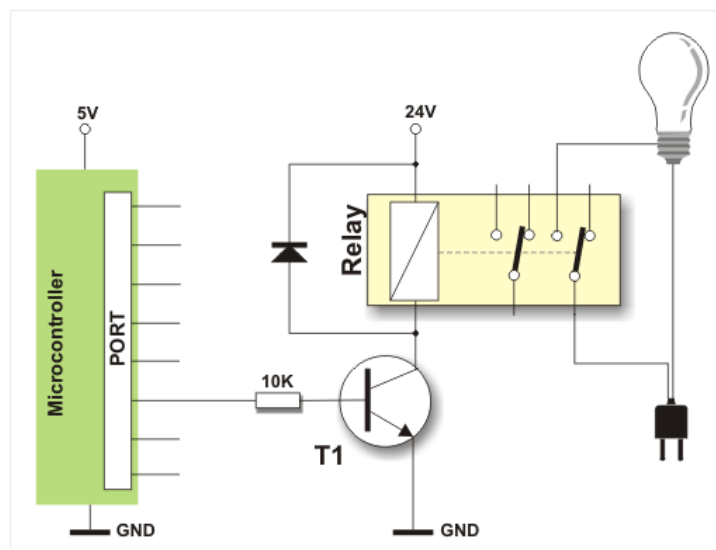
A relay is an electrical switch that opens and closes under the control of another electrical circuit. It is therefore connected to output pins of the microcontroller and used to turn on/off high-power devices such as motors, transformers, heaters, bulbs, etc. These devices are almost always placed away from the boards sensitive components. There are various types of relays, but all of them operate in the same way. When a current flows

through the coil, the relay is operated by an electromagnet to open or close one or many sets of contacts. Similar to optocouplers, there is no galvanic connection (electrical contact) between input and output circuits. Relays usually demand both higher voltage and current to start operation but there are also miniature ones that can be activated by a low current directly obtained from a microcontroller pin.



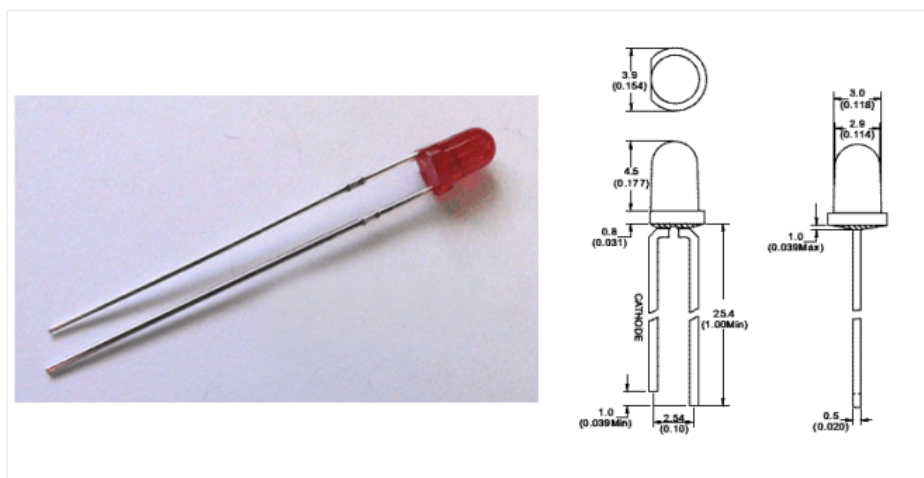
This figure shows the most commonly used solution.

In order to prevent the appearance of high voltage self-induction caused by a sudden stop of current flow through the coil, an inverted polarized diode is connected in parallel to the coil. The purpose of this diode is to “cut off” the voltage peak.



LED DIODES

You probably know all you need to know about LED diodes, but we should also think of the younger generations...How to destroy a LED?! Well...Very simple.



Quick burning

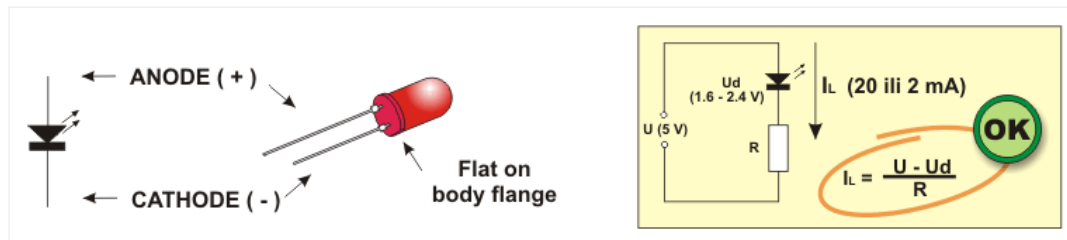
Like any other diode, LEDs have two ends an anode and a cathode. Connect it properly to a power supply voltage. The diode will happily emit light. Turn it upside down and apply the same power supply voltage (even for a moment). It will not emit light- NEVER AGAIN!

Slow burning

There is a nominal, i.e. maximum current determined for every LED which should not be exceeded. If it happens, the diode will emit more intensive light, but not for a long time!

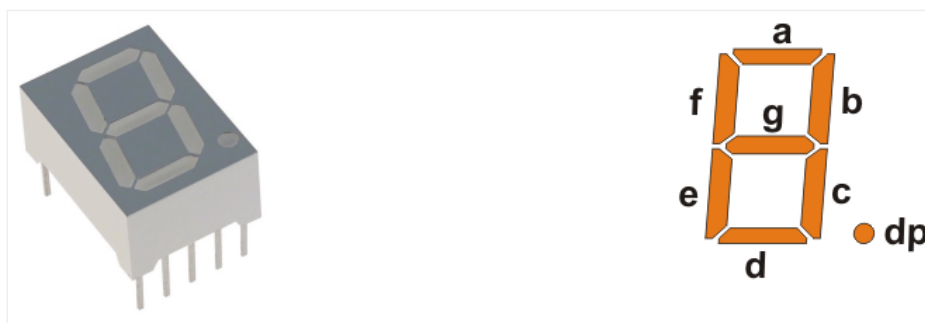
Something to remember

Similar to the previous example, all you need to do is to discard a current limiting resistor shown below. Depending on power supply voltage, the effect might be spectacular!



LED DISPLAY

Basically, LED display is nothing more than several LEDs moulded in the same plastic case. There are many types of displays composed of several dozens of built in diodes which can display different symbols. The most commonly used is so called 7-segment display. It is composed of 8 LEDs- 7 segments are arranged as a rectangle for symbol displaying and there is an additional segment for decimal point displaying. In order to simplify connection, anodes or cathodes of all diodes are connected to the common pin so that there are common anode displays and common cathode displays, respectively. Segments are marked with the letters from a to g, plus dp, as shown in figure below. On connecting, each diode is treated separately, which means that each must have its own current limiting resistor.

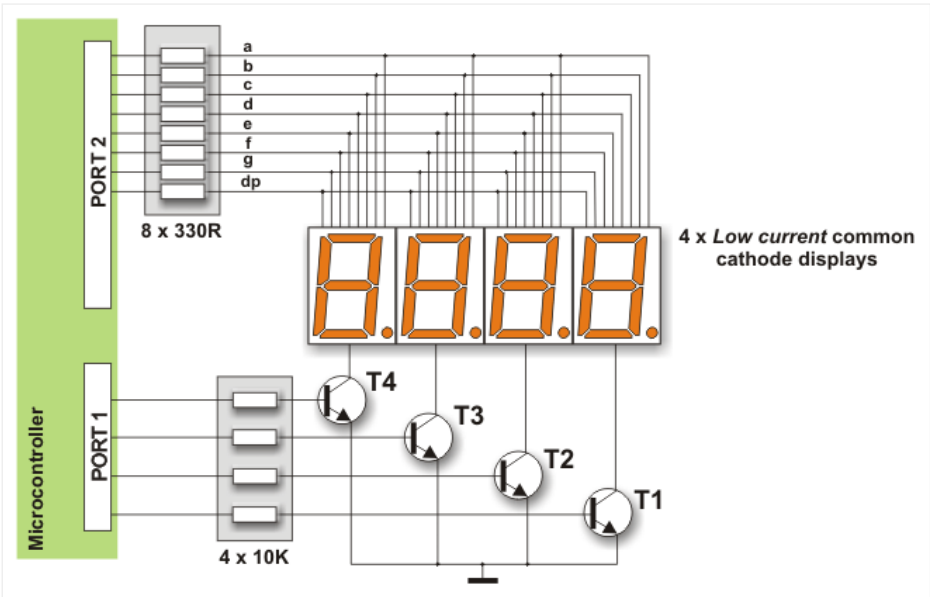


Here are a few important things that one should pay attention to when buying LED displays:

- Depending on whether anodes or cathodes are connected to the common pin, there are common anode displays and common cathode displays. The figure above shows a common anode display. Looking at physical features, there is no difference between these displays at all so it is recommended to check carefully prior installation which of them is in use;
- For each microcontroller pin, there is a maximum current limitation it can receive or give. Because of this, if several displays are connected to the microcontroller it is recommended to use so called *Low current* LEDs using only 2mA for operation; and
- Display segments are usually marked with the letters from a to g, but there is no fast rule indicating to which micro controller pins they should be connected. For this reason it is very important to check connecting prior to commencing program writing or designing a device.

Displays connected to the microcontroller usually occupy a large number of valuable I/O pins, which can be a big problem especially when it is needed to display multi-digit numbers. The problem is more than obvious if, for example, it is needed to display two 6-digit numbers (a simple calculation shows that 96 output pins are needed in this case)! This problem has a solution called MULTIPLEXING.

Here is how an optical illusion based on the same operating principle as a film camera is made. Only one digit at a time is active, but they change their state so quickly that one gets impression that all digits of a number are active simultaneously.

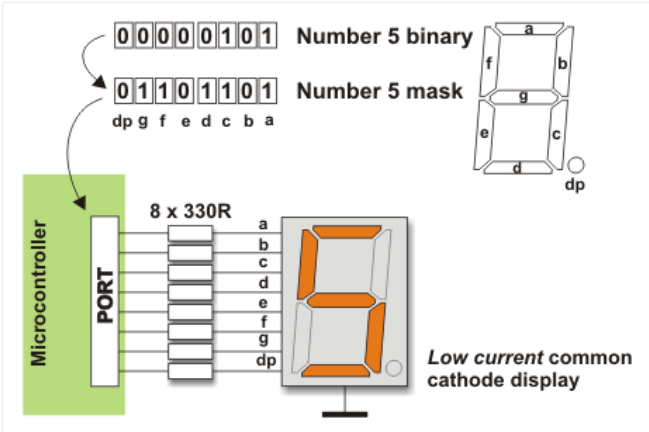


Here is an explanation on the figure above. First a byte representing units is applied on a microcontroller port and a transistor T1 is activated simultaneously. After a while, the transistor T1 is turned off, a byte representing tens is applied on a port and transistor T2 is activated. This process is being cyclically repeated at high speed for all digits and corresponding transistors.

A disappointing fact which indicates that the microcontroller is just a kind of miniature computer designed to understand only the language of zeros and ones is fully expressed when displaying any digit. Namely, the microcontroller does not know what units, tens or hundreds are, nor what ten digits we are used to look like. Therefore, each number to be displayed must go through the following procedure:

First of all, in a particular subroutine a multi-digit number must be split into units, tens etc. Then, these must be stored in special bytes each. Digits get recognizable format by performing “masking”. In other words, a binary format of each digit is replaced by a different combination of bits using a simple subroutine. For example, the digit 8 (0000 1000) is replaced by binary number 0111 1111 in order to activate all LEDs displaying digit 8. The only diode remaining inactive in this case is reserved for the decimal point.

If a microcontroller port is connected to the display in a way that bit 0 activates segment “a”, bit 1 activates segment “b”, bit 2 segment “c” etc., then the table below shows the mask for each digit.



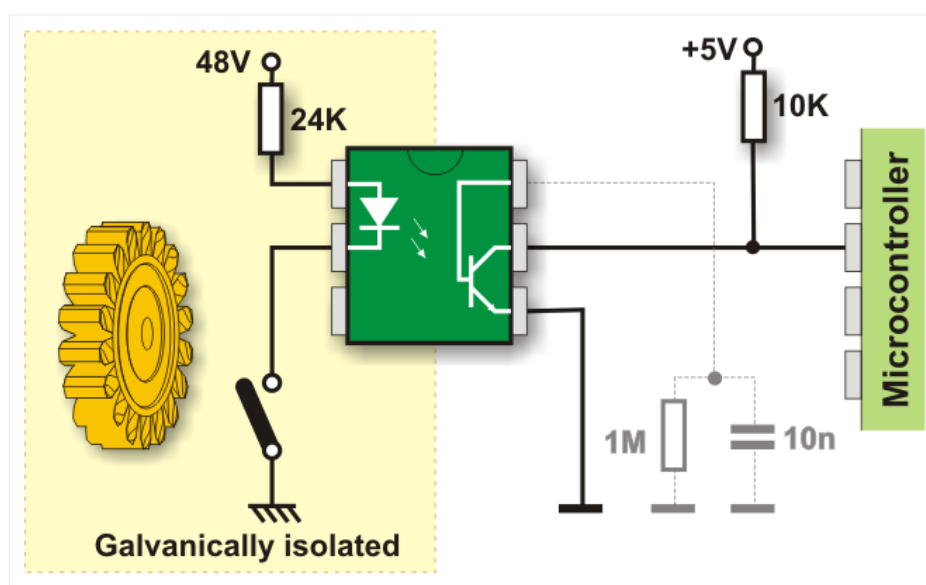
DIGITS TO DISPLAY	DISPLAY SEGMENTS							
	dp	a	b	c	d	e	f	g
0	0	1	1	1	1	1	1	0
1	0	0	1	1	0	0	0	0
2	0	1	1	0	1	1	0	1
3	0	1	1	1	1	0	0	1
4	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1
6	0	1	0	1	1	1	1	1
7	0	1	1	1	0	0	0	0
8	0	1	1	1	1	1	1	1
9	0	1	1	1	1	0	1	1

In addition to digits from 0 to 9, there are some letters- A, C, E, J, F, U, H, L, b, c, d, o, r, t- that can be also displayed by means of the appropriate masking.

In the event that the common anode displays are used, all ones contained in the previous table should be replaced by zeros and vice versa. Additionally, NPN transistors should be used as drivers as well.

OPTOCOUPLER

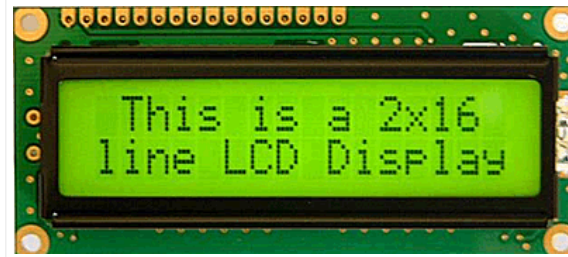
An optocoupler is a device commonly used to galvanically separate microcontroller electronics from any potentially dangerous current or voltage in its surroundings. Optocouplers usually have one, two or four light sources (LED diodes) on their input while on their output, opposite to diodes, there is the same number of elements sensitive to light (phototransistors, photo-thyristors or photo-triacs). The point is that an optocoupler uses a short optical transmission path to transfer a signal between elements of circuit, while keeping them electrically isolated. This isolation makes sense only if diodes and photo-sensitive elements are separately powered. In this way, the microcontroller and expensive additional electronics are completely protected from high voltage and noises which are the most common cause of destroying, damaging or unstable operation of electronic devices in practice. The most frequently used optocouplers are those with phototransistors on their outputs. Additionally, optocouplers with internal base-to-pin 6 connection (there are also optocouplers without it), the base may be left unconnected.



The R/C network represented by the broken line in the figure above denotes optional connection which lessens the effects of noises by eliminating very short pulses.

LCD DISPLAY

This component is specifically manufactured to be used with microcontrollers, which means that it cannot be activated by standard IC circuits. It is used for displaying different messages on a miniature liquid crystal display. The model described here is for its low price and great capabilities most frequently used in practice. It is based on the HD44780 microcontroller (*Hitachi*) and can display messages in two lines with 16 characters each. It displays all the letters of alphabet, Greek letters, punctuation marks, mathematical symbols etc. In addition, it is possible to display symbols made up by the user. Other useful features include automatic message shift (left and right), cursor appearance, LED backlight etc.

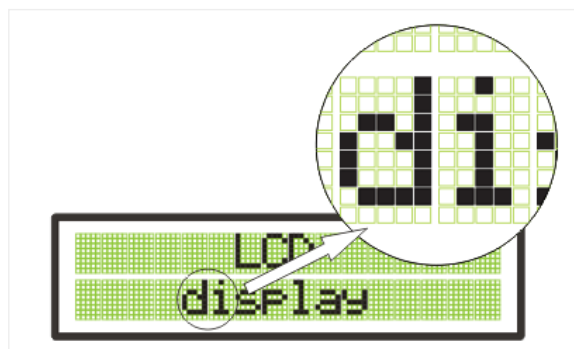


Along one side of a small printed board there are pins used for connecting to the microcontroller. There are in total of 14 pins marked with numbers (16 if the backlight is built in). Their function is described in the table below:

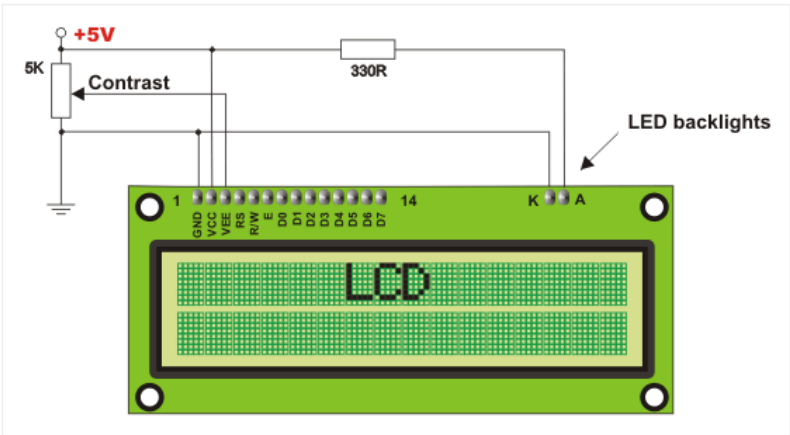
FUNCTION	PIN NUMBER	NAME	LOGIC STATE	DESCRIPTION
Ground	1	Vss	–	0V
Power supply	2	Vdd	–	+5V
Contrast	3	Vee	–	0 – Vdd
Control of operating	4	RS	0 1	D0 – D7 are interpreted as commands D0 – D7 are interpreted as data
	5	R/W	0 1	Write data (from controller to LCD) Read data (from LCD to controller)
	6	E	0 1 From 1 to 0	Access to LCD disabled Normal operating Data/commands are transferred to LCD
Data / commands	7	D0	0/1	Bit 0 LSB
	8	D1	0/1	Bit 1
	9	D2	0/1	Bit 2
	10	D3	0/1	Bit 3
	11	D4	0/1	Bit 4
	12	D5	0/1	Bit 5
	13	D6	0/1	Bit 6
	14	D7	0/1	Bit 7 MSB

LCD screen

The LCD screen consists of two lines with 16 characters each. Every character consists of 5×8 or 5×11 dot matrix. This book covers the 5×8 character display, which is indeed the most commonly used.



Display contrast depends on the power supply voltage and whether messages are displayed in one or two lines. For this reason, varying voltage 0-Vdd is applied on the pin marked as Vee. Trimmer potentiometer is usually used for that purpose. Some LCD displays have built in backlight (blue or green diodes). When used during operation, a current limiting resistor should be serially connected to one of the pins for backlight (similar to LED diodes).



If there are no characters displayed or if all of them are dimmed when the display is switched on, the first thing that should be done is to check the potentiometer for contrast adjustment. Is it properly adjusted? The same applies if the mode of operation has been changed (writing in one or two lines).

LCD Memory

LCD display contains three memory blocks:

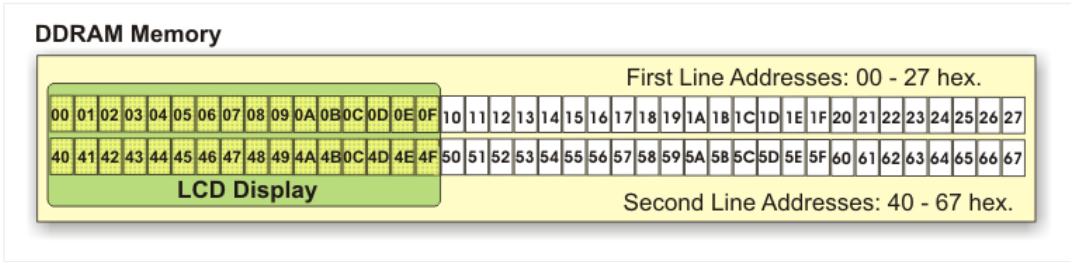
- DDRAM – Display Data RAM;
- CGRAM – Character Generator RAM; and
- CGROM – Character Generator ROM.

DDRAM Memory

DDRAM memory is used for storing characters that should be displayed. The size of this memory is sufficient for storing 80 characters. Some memory locations are directly connected to the characters on display.

It works quite simply: it is enough to configure the display to increment addresses automatically (shift right) and set the starting address for the message that should be displayed (for example 00 hex).

After that, all characters sent through lines D0-D7 will be displayed as a message format we are used to- from left to right. In this very case, displaying starts from the first field of the first line because the address is 00 hex. If more than 16 characters are sent then all of them will be memorized, but only the first sixteen characters will be visible. In order to display the rest of them, a shift command should be used. Virtually, everything looks as if the LCD display is a window which shifts left-right over memory locations containing different characters. In reality, this is how the effect of message shifting on the screen has been created.



If the cursor is on, it appears at the location which is currently addressed. In other words, when a character appears at the cursor position, it will automatically move to the next addressed location.

This is a sort of RAM memory so data can be written to and read from it, but its contents is irretrievably lost upon the power goes off.

CGROM Memory

CGROM memory contains the default character map with all characters that can be displayed on the screen. Each character is assigned to one memory location:

		4 higher bits of address																	
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111		
4 lower bits of address	xxxx0000	CG RAM (1)			0	Q	P	`	P					-	9	3	α	p	
	xxxx0001	(2)			!	1	A	Q	a	q				.	7	4	ä	q	
	xxxx0010	(3)			"	2	B	R	b	r				'	イ	ツ	×	β	θ
	xxxx0011	(4)			#	3	C	S	c	s				」	ウ	〒	ε	∞	
	xxxx0100	(5)			\$	4	D	T	d	t				\	エ	ト	†	μ	Ω
	xxxx0101	(6)			%	5	E	U	e	u				.	オ	ナ	1	5	Ü
	xxxx0110	(7)			&	6	F	V	f	v				ヲ	カ	ニ	ヨ	ρ	Σ
	xxxx0111	(8)			'	7	G	W	g	w				ア	キ	ヌ	ラ	g	π
	xxxx1000	(1)			(	8	H	X	h	x				イ	ク	キ	リ	フ	Σ
	xxxx1001	(2)			)	9	I	Y	i	y				ウ	ケ	ル	リ	ウ	4
	xxxx1010	(3)			*	:	J	Z	j	z				エ	コ	ン	レ	j	チ
	xxxx1011	(4)			+	;	K	[	k	{				オ	サ	ヒ	ロ	×	π
	xxxx1100	(5)			,	<	L	¥	l	l				ハ	シ	フ	ワ	Φ	π
	xxxx1101	(6)			-	=	M	]	m	}				ユ	ズ	ン	ト	÷	
	xxxx1110	(7)			.	>	N	^	n	→				ヨ	セ	ホ	°	π	
	xxxx1111	(8)			/	?	O	_	o	€				ッ	ソ	マ	°	Ö	■

The addresses of CGROM memory locations match the characters of ASCII. If the program being currently executed encounters a command “send character P to port” then the binary value 0101 0000 appears on the port. This value is the ASCII equivalent to the character P. It is then written to LCD, which results in displaying the symbol from the 0101 0000 location of CGROM. In other words, the character “P” is displayed. This applies to all letters of the alphabet (capitals and small), but not to the numbers!

As seen on the previous map, addresses of all digits are pushed forward by 48 relative to their values (digit 0 address is 48, digit 1 address is 49, digit 2 address is 50 etc.). Accordingly, in order to display digits correctly it is necessary to add a decimal number 48 to each of them prior to sending them to LCD.

What is ASCII? From their inception till today, computers can recognize only numbers, but not letters. It means that all data a computer swaps with a peripheral device has a binary format even though the same is recognized by the man as letters (The keyboard is an excellent example)! It's as simple as that- every character matches the unique combination of zeroes and ones. ASCII is character encoding based on the English alphabet. ASCII code specifies a correspondence between standard character symbols and their numerical equivalents.

LCD Basic Commands

All data transferred to LCD through the outputs D0-D7 will be interpreted as a command or a data, which depends on the pin RS logic state:

RS = 1 – Bits D0 – D7 are addresses of the characters to be displayed. LCD processor addresses one character from the character map and displays it. The DDRAM address specifies the location on which the character is to be displayed. This address is defined prior character transfer or the address of the previously transferred character is automatically incremented.

RS = 0 – Bits D0 – D7 are commands which determine display mode.

The commands recognized by the LCD are listed in table below:

COMMAND	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	EXECUTION TIME
Clear display	0	0	0	0	0	0	0	0	0	1	1.64ms
Cursor home	0	0	0	0	0	0	0	0	1	x	1.64ms
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	40uS
Display on/off control	0	0	0	0	0	0	1	D	U	B	40uS
Cursor/Display Shift	0	0	0	0	0	1	D/C	R/L	x	x	40uS
Function set	0	0	0	0	1	DL	N	F	x	x	40uS
Set CGRAM address	0	0	0	1	CGRAM address						40uS
Set DDRAM address	0	0	1	DDRAM address							40uS
Read "BUSY" flag (BF)	0	1	BF	DDRAM address							–
Write to CGRAM or DDRAM	1	0	D7	D6	D5	D4	D3	D2	D1	D0	40uS
Read from CGRAM or DDRAM	1	1	D7	D6	D5	D4	D3	D2	D1	D0	40uS

1 I/D 1 = Increment (by 1)	R/L 1 = Shift right
2 0 = Decrement (by 1)	0 = Shift left
3	
4 S 1 = Display shift on	DL 1 = 8-bit interface
5 0 = Display shift off	0 = 4-bit interface
6	
7 D 1 = Display on	N 1 = Display in two lines
8 0 = Display off	0 = Display in one line
9	
10 U 1 = Cursor on	F 1 = Character format 5x10 dots
11 0 = Cursor off	0 = Character format 5x7 dots
12	
13 B 1 = Cursor blink on	D/C 1 = Display shift
14 0 = Cursor blink off	0 = Cursor shift

What is Busy flag ?

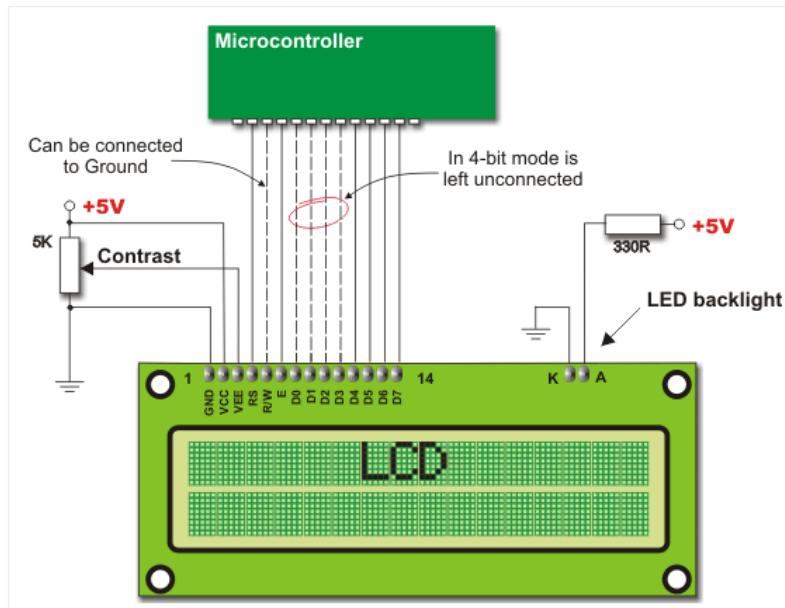
Compared to the microcontroller, the LCD is an extremely slow component. Because of this, it was necessary to provide a signal which would, upon command execution, indicate that the display is ready for the next piece of data. That signal, called the *busy flag*, can be read from the line D7. When the voltage on this line is 0V (BF=0), the display is ready to receive new data.

LCD Connecting

Depending on how many lines are used for connecting the LCD to the microcontroller, there are 8-bit and 4-bit LCD modes. The appropriate mode is selected at the beginning of the operation in this process called "initialization". 8-bit LCD mode uses outputs D0-D7 to transfer data as explained on the previous page.

The main purpose of 4-bit LED mode is to save valuable I/O pins of the microcontroller. Only 4 higher bits (D4-D7) are used for communication, while others may be unconnected. Each piece of data is sent to the LCD in two

steps- four higher bits are sent first (normally through the lines D4-D7) and four lower bits are sent afterwards. Initialization enables the LCD to link and interpret received bits correctly.



Data is rarely read from the LCD (it is mainly transferred from the microcontroller to LCD) so it is often possible to save an extra I/O pin by simply connecting R/W pin to the Ground. Such saving has its price. Messages will be normally displayed, but it will not be possible to read the busy flag since it is not possible to read the display as well. Fortunately, there is a simple solution. After sending a character or a command it is important to give the LCD enough time to do its job. Owing to the fact that the execution of the slowest command lasts for approximately 1.64mS, it will be sufficient to wait approximately 2mS for LCD.

LCD Initialization

The LCD is automatically cleared when powered up. It lasts for approximately 15mS. After this, display is ready for operation. The mode of operation is set by default. It means that:

1. Display is cleared.

2. Mode

DL = 1 Communication through 8-bit interface

N = 0 Messages are displayed in one line

F = 0 Character font 5 x 8 dots

3. Display/Cursor on/off

D = 0 Display off

U = 0 Cursor off

B = 0 Cursor blink off

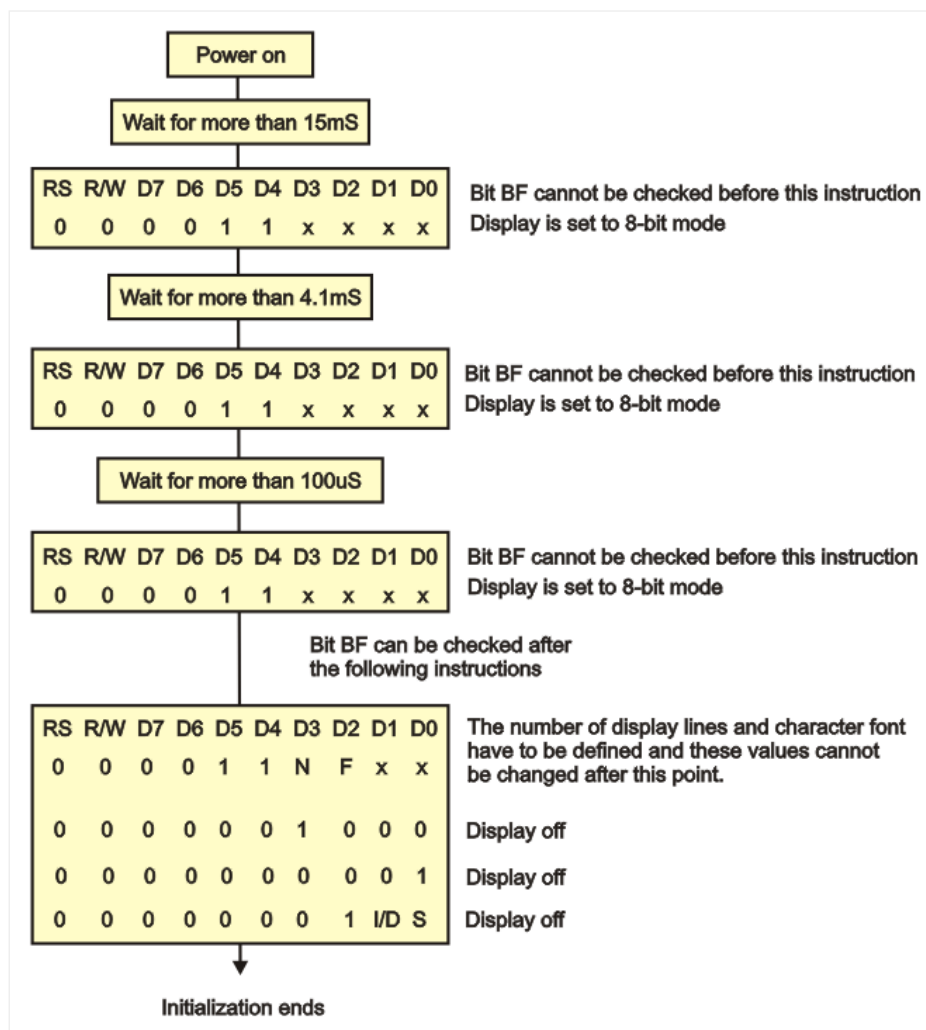
4. Character entry

ID = 1 Displayed addresses are automatically incremented by 1

S = 0 Display shift off

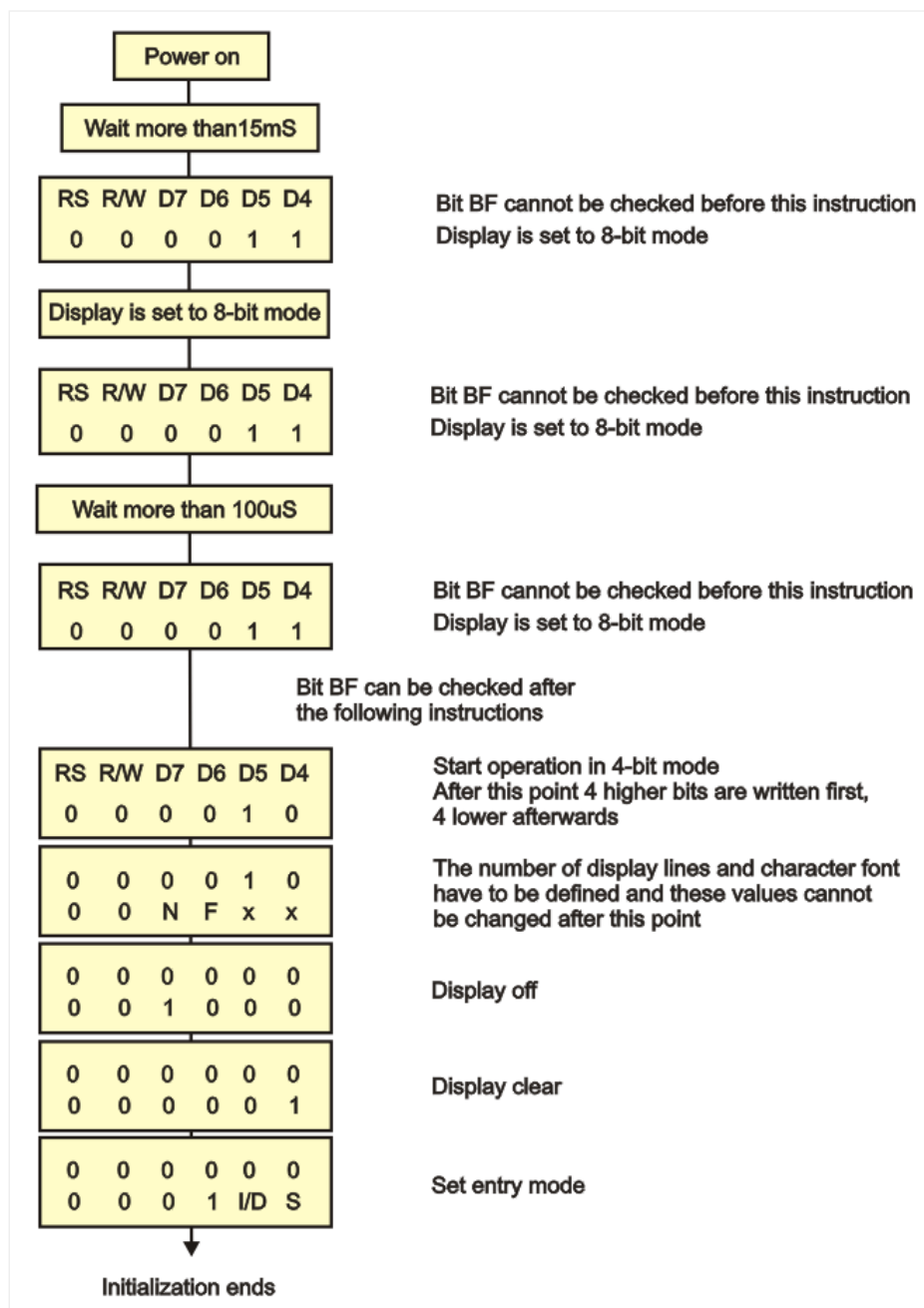
Automatic reset is mostly done without any problems. Mostly, but not always! If for any reason power supply voltage does not reach full value within 10mS, display will start performing completely unpredictably. If voltage supply unit is not able to meet that condition or if it is needed to provide completely safe operation, the process of initialization is applied. Initialization, among other things, causes a new reset enabling display to operate normally.

Refer to the figure below for the procedure on 8-bit initialization:



It is not a mistake! In this algorithm, the same value is transferred three times in a row.

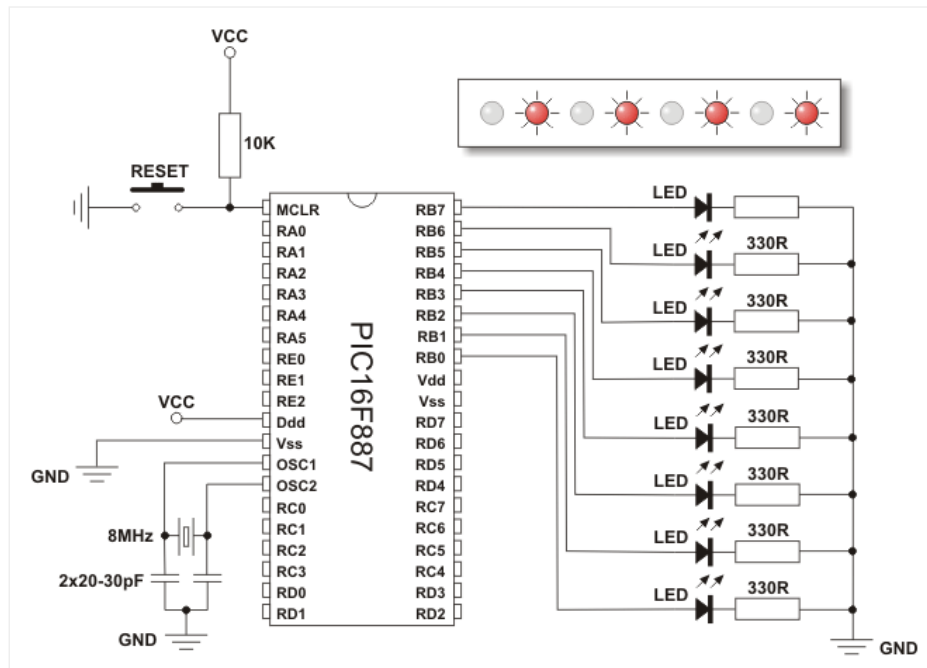
In case of 4-bit initialization, the procedure is as follows:



EXAMPLE 1

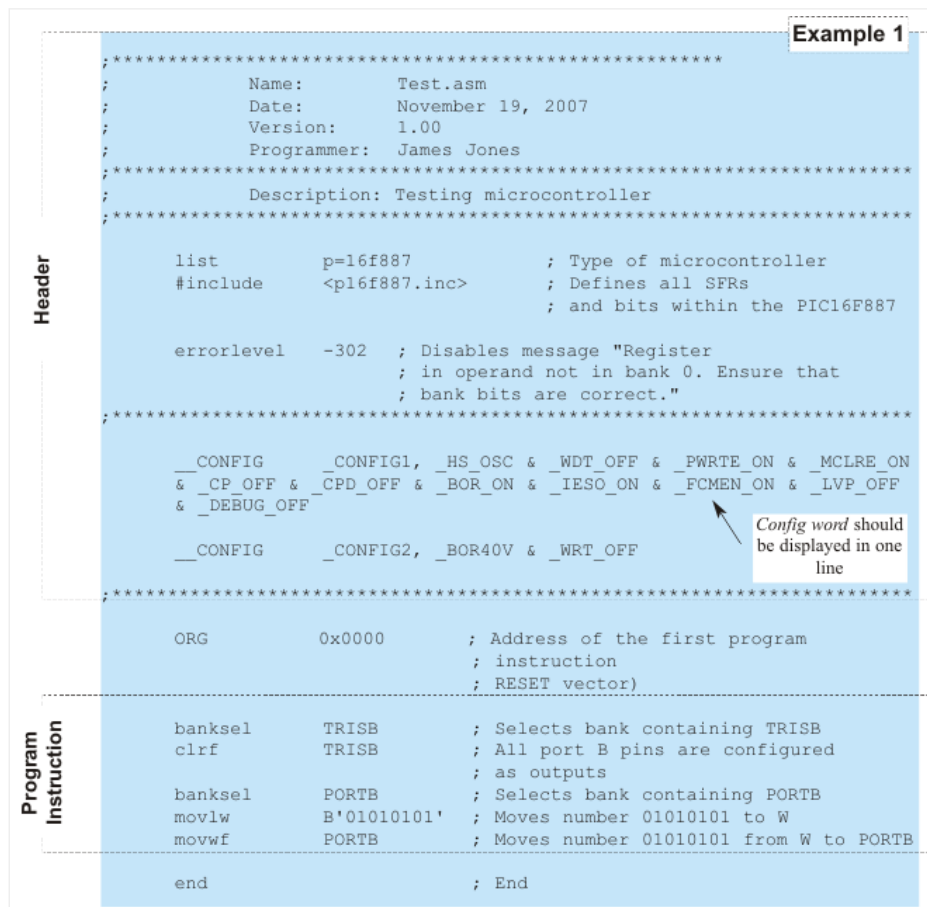
Writing header and configuring I/O pins

The only purpose of this program is to turn on a few LED diodes on port B. It is nothing special. Anyway, use this example to study what a real program looks like. The figure below shows a connection scheme, while the program is on the next page.



When switching on, every other LED diode on the port B emits light. That is enough to indicate that the microcontroller is properly connected and operates normally.

This example gives the description of a correctly written header and a few initial directives. They represent a part of the program used in all programs described in this book. To skip repetitiveness, it will not be written in the following examples, but is considered to be at the beginning of every program (marked as a “Header”).



The purpose of the header and initial directives is briefly described below.

Header:

The header is placed at the beginning of the program and gives basic information in the form of comments (name of the program, release date etc.). Don't be deluded into thinking that after a few months you will know what that

program is about and why it is saved in your computer.

Initial directives:

```
1 list p=16f887
```

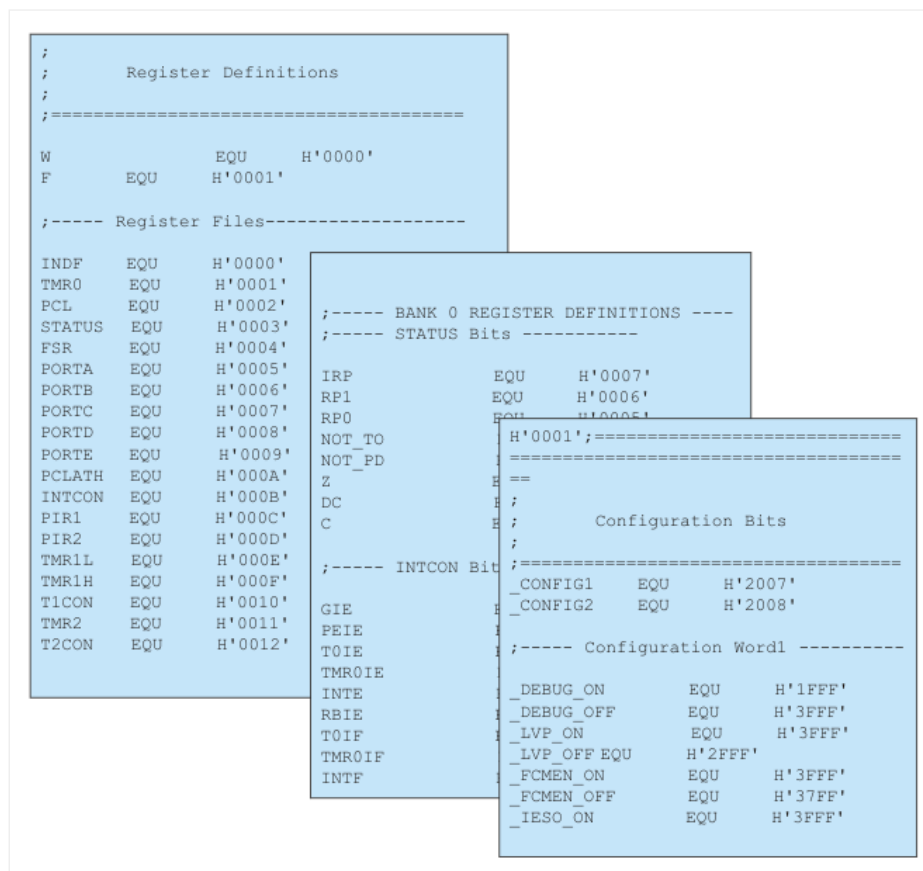
This directive defines processor to execute a program.

```
1 #include <p16f887.inc>
```

It enables the compiler to access the document p16f887.inc (If you have MPLAB installed, it is placed by default on C:\Program files\Microchip\MPASM Suite). Every SFR register contained in this document, as well as every bit, has its own name and address. If the program reads for example:

```
1 bsf INTCON, GIE
```

It means that the GIE bit of the INTCON register should be set. Instruction, as such, makes no sense to the compiler. It has to access the “.inc” document in order to know that the seventh bit of the SFR at the address 000B hex should be set.



```
1 errorlevel -302
```

This is a “cosmetic” directive which disables the irritating message “Register in operand not in ...” to appear at the end of every compiling process. It is not necessary, but useful.

```
1 __config
```

This directive is used to include config word in the program upon compiling. It is not necessary because the same operation is performed by software for loading program into chip. However, do you have any idea which software will be used by the end user? What options will be set by default? You are the end user?! Do you know which program you will be using for MCU programming next year? Make life easier for yourself, take this directive as a necessary one and include it in your program.

EXAMPLE 2

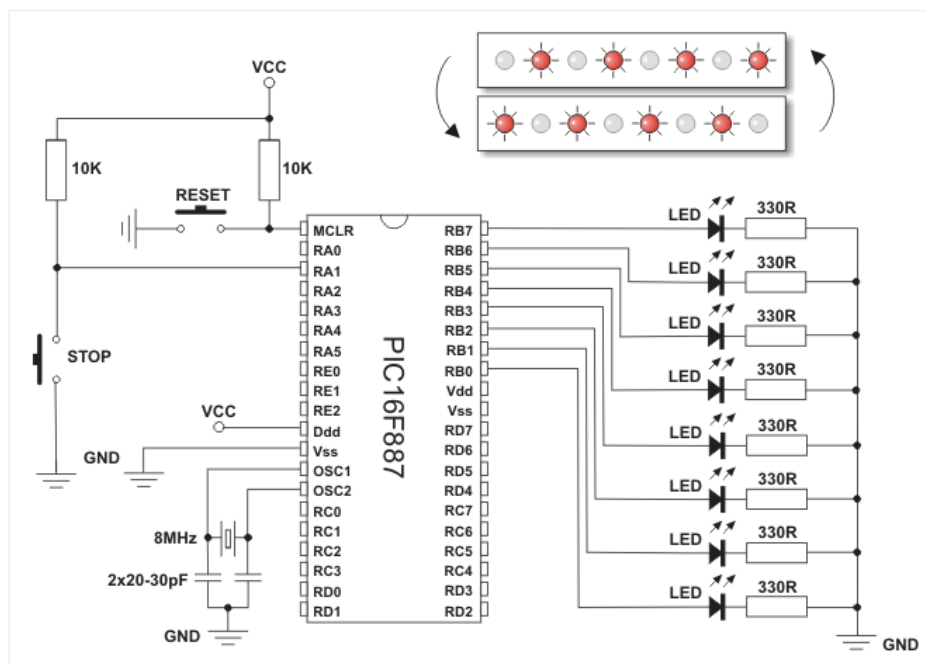
Using program loop and internal oscillator LFINTOSC

This is a continuation of the previous example, but deals with a bit more complicated problem... The idea is to make the LED diodes on the port B blink. A simple thing at first glance! It is enough to periodically change logic state on the port B. In this case, numbers 01010101 and 10101010 are selected to change in the following way:

1. Set binary combination 01010101 on port B;
2. Remain in loop1;
3. Replace existing bits combination on port B with 10101010;
4. Remain in loop2; and
5. Return to the step 1 and repeat the whole procedure.

Do you know how fast this should be done? It would be possible to observe changes on port B only if, apart from the delays provided in loop1 and loop2, the whole process is slowed down approximately 250 times more. Because of this, the microcontroller uses internal oscillator LFINTOSC with the frequency of 31kHz instead of the external oscillator with quartz crystal (8MHz).

You have noticed that the clock signal source has changed “on the fly”. If you want to make sure of it, remove quartz crystal prior to switching the microcontroller on. What will happen? The microcontroller will not start operating because the config word loaded with the program requires the use of the crystal on switching on. If you remove the crystal later during the operation, it will not affect the microcontroller at all!



Source Code

```

1 ;*****
2 ;
3 ;***** Header *****
4 ;***** DEFINING VARIABLES *****
5     cblock 0x20          ; Block of variables starts at address 20h
6     counter1            ; Variable "counter1" at address 20h
7     endc
8 ;*****
9     org 0x0000          ; Address of the first program instruction
10
11     banksel OSCCON       ; Selects memory bank containing
12                          ; register OSCCON
13     bcf OSCCON,6         ; Selects internal oscillator LFINTOSC with
14     bcf OSCCON,5         ; the frequency of 31KHz
15     bcf OSCCON,4
16     bsf OSCCON,0        ; Microcontroller uses internal oscillator
17
18     banksel TRISB        ; Selects bank containing register TRISB
19     clrf TRISB           ; All port B pins are configured as outputs
20     banksel PORTB        ; Selects bank containing register PORTB
21
22 loop
23     movlw B'01010101'    ; Binary number 01010101 is written to W
24     movwf PORTB          ; Number is moved to PORTB
25     movlw h'FF'          ; Number hFF is moved to W
26     movwf counter1       ; Number is moved to variable "counter1"

```

```

27
28 loop1
29     decfsz   counter1      ; Variable "counter1" is decremented by 1
30     goto     loop1        ; If result is 0, continue. If not,
31                             ; remain in loop1
32
33     movlw    B'10101010'   ; Binary number 10101010 is moved to W
34     movwf    PORTB        ; Number is moved to PORTB
35     movlw    h'FF'         ; Number hFF is moved to W
36     movwf    counter1     ; Number is moved to variable "counter1"
37
38 loop2
39     decfsz   counter1      ; Variable "counter1" is decremented by 1
40     goto     loop2        ; If result is 0, continue. If not,
41                             ; remain in loop2
42
43     goto     loop         ; Go to label loop
44     end                          ; End of program

```

EXAMPLE 3

Using nested loop

The connection scheme is again the same. To make this a bit more interesting, a different combination of port B bits change each other. And, that's not all of course. As seen from the previous two examples, the microcontroller is very fast and often, it needs to be slowed down. The use of the built-in oscillator LF, as in example 2, is the last measure that should be applied. The problem is more often solved by using nested loops in a program. In this example, the variable "counter1" is decremented 255 times by 1 in the shorter loop1. Prior to leaving this loop, the program will countdown 255 times from 255 to 0. It means that between only two LED diode's blink on the port, there are 255×255 pulses coming from the quartz oscillator. Precisely speaking, the number of pulses amounts to approximately 196 000 since it also takes some time to execute jump instructions and decrement instructions. Yes, it's true, the microcontroller mostly waits and does nothing...

Source Code

```

1  ;***** Header *****
2  ;***** DEFINING VARIABLES *****
3
4      cblock    0x20        ; Block of variables starts at address 20h
5      counter1      ; Variable "counter1" at address 20h
6      counter2      ; Variable "counter2" at address 21h
7      endc
8  ;*****
9      org        0x0000    ; Address of the first program instruction
10
11     banksel     TRISB     ; Selects bank containing register TRISB
12     clrf        TRISB    ; Clears TRISB
13
14     banksel     PORTB     ; Selects bank containing register PORTB
15 loop
16     movlw       B'11110000' ; Binary number 11110000 is moved to W
17     movwf       PORTB      ; Number is moved to PORTB
18     movlw       h'FF'      ; Number hFF is moved to W
19     movwf       counter2   ; Number is moved to variable "counter2"
20
21 loop2
22     movlw       h'FF'      ; Number hFF is moved to W
23     movwf       counter1   ; Number is moved to "counter1"
24 loop1
25     decfsz      counter1   ; Decrements "counter1" by 1. If result is 0
26     goto        loop1     ; skip next instruction
27
28     decfsz      counter2   ; Decrements "counter2" by 1. If result is 0
29     goto        loop2     ; skip next instruction
30
31     movlw       B'00001111' ; Binary number 00001111 is moved to W
32     movwf       PORTB      ; Number is moved to PORTB
33     movlw       h'FF'      ; Number hFF is moved to W
34     movwf       counter2   ; Number is moved to variable "counter2"
35
36 loop4
37     movlw       h'FF'      ; Number hFF is moved to W
38     movwf       counter1   ; Number is moved to variable "counter1"
39 loop3
40     decfsz      counter1   ; Decrements "counter1" by 1. If result is 0
41     goto        loop3     ; skip next instruction
42     goto        loop3
43     decfsz      counter2   ; Decrements "counter2" by 1. If result is 0
44     goto        loop4     ; skip next instruction
45     goto        loop      ; Jump to label loop
46     end                          ; End of program

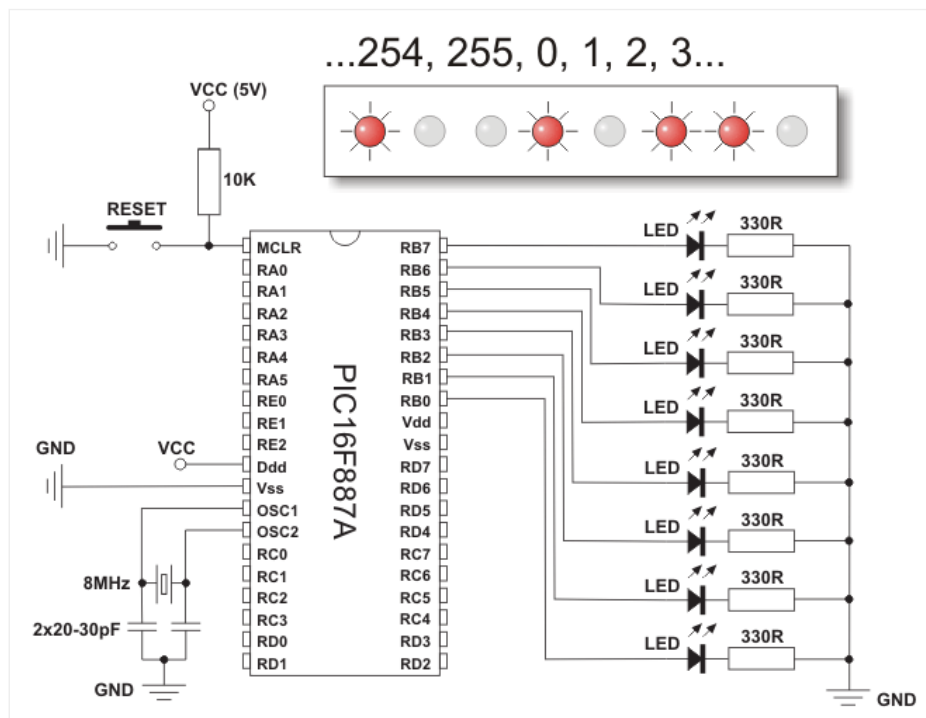
```

Example 4

Using timer TMR0 and Interrupts

If you have read the previous example, you would have noticed a disadvantage of providing delays using loops. In all these cases, the microcontroller is “captive” and does nothing. It simply waits for some time to pass. Such wasting of time is an unacceptable luxury and some other method should be applied.

Do you remember the story about the timers? About interrupts? This example makes links between them in a practical way. The schematic is still the same as well as the challenge. It is necessary to provide delay long enough to notice changes on a port. This time, the timer TMR0 with the assigned prescaler is used for that purpose. Interrupt occurs on every timer register overflow and interrupt routine increments the number in port B by 1. The whole procedure is performed “behind the scenes” of the whole process, which enables the microcontroller to do other things.



Pay attention to a few details:

- Even though it is unnecessary in this case, the contents of the most important registers (W, STATUS and PCLATH) must be saved at the beginning of the interrupt routine;
- Interrupt causes the appropriate flag bit to be automatically set and the GIE bit to be automatically cleared. At the end of the interrupt routine, do not forget to return these bits to the state they had prior to the interrupt occurring; and
- At the end of the interrupt routine, important registers should be given the original content.

Source Code

```

1 ;***** Header *****
2 ;***** DEFINING VARIABLES *****
3
4     cblock      0x20          ; Block of variables starts at address 20h
5     w_temp      ; Variable at address 20h
6     pclath_temp ; Variable at address 21h
7     status_temp ; Variable at address 22h
8     endc
9
10 ;***** START OF PROGRAM *****
11     org      0x0000          ; Address of the first program instruction
12     goto     main           ; Go to label "main"
13
14 ;***** INTERRUPT ROUTINE *****
15     org      0x0004          ; Interrupt vector
16     movwf    w_temp          ; Saves value in register W
17     movf     STATUS, W       ; Saves value in register STATUS

```

```

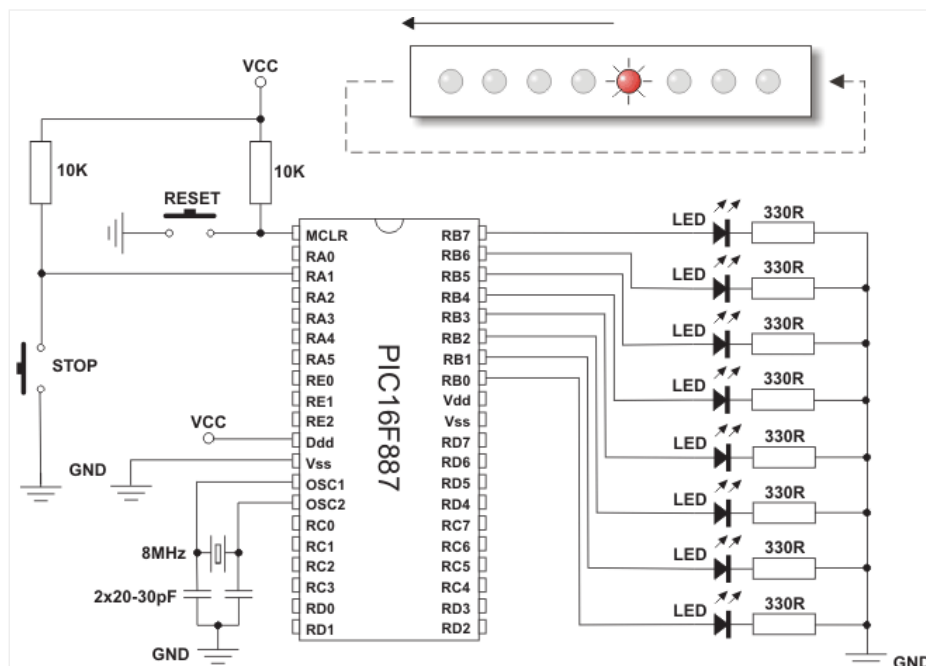
18      movwf    status_temp
19      movf     PCLATH          ; Saves value in register PCLATH
20      movwf    pclath_temp
21
22      banksel   PORTB          ; Selects bank containing PORTB
23      incf     PORTB          ; Increments register PORTB by 1
24
25      banksel   INTCON         ; Selects bank containing INTCON
26      bcf      INTCON,TMR0IF   ; Clears interrupt flag TMR0IF
27
28      movf     pclath_temp,w    ; PCLATH is given its original content
29      movwf    PCLATH
30      movf     status_temp,w   ; STATUS is given its original content
31      movwf    STATUS
32      swapf    w_temp,f        ; W is given its original content
33      swapf    w_temp,w
34
35      bsf      INTCON,GIE      ; Global interrupt enabled
36      retfie   ; Return from interrupt routine
37
38 ;***** MAIN PROGRAM *****
39 main
40
41      banksel   ANSEL          ; Bank containing register ANSEL
42      clrf     ANSEL          ; Clears registers ANSEL and ANSELH
43      clrf     ANSELH
44
45      banksel   TRISB          ; Selects bank containing register TRISB
46      clrf     TRISB          ; All port B pins are configured as outputs
47
48      banksel   OPTION_REG     ; Bank containing register OPTION_REG
49      bcf      OPTION_REG,T0CS ; TMR0 counts pulses from oscillator
50      bcf      OPTION_REG,PSA  ; Prescaler is assign to timer TMR0
51
52      bsf      OPTION_REG,PS0  ; Prescaler rate is 1:256
53      bsf      OPTION_REG,PS1
54      bsf      OPTION_REG,PS2
55
56      banksel   INTCON         ; Bank containing register INTCON
57      bsf      INTCON,TMR0IE   ; TMR0 interrupt overflow enabled
58      bsf      INTCON,GIE      ; Global interrupt enabled
59
60      banksel   PORTB          ; Bank containing register PORTB
61      clrf     PORTB          ; Clears port B
62 loop
63      goto     loop           ; Remain here
64      end                ; End of program

```

Example 5

Using subroutine, using push-buttons

In the previous examples the microcontroller executes the program without being influenced in any way its surrounding. In practice, devices operating in this way are very rare (for example, simple neon signs). You guess, among other components, input pins will also be used in this example. There is a schematic in the figure below, while the program is on the next page. Everything is still very simple.



At the beginning of the program, immediately upon defining variables, the microcontroller pins* are configured by using registers TRISA and TRISB.

In the main program, one bit on port B is set first. Then the contents of this register is constantly moved by one place to the left (instruction `rlf PORTB`). It gives us the impression that the lit LED diodes is moving. To make it visible, the whole process must be slow enough. Press on the push-button “STOP” stops the movement and the program remains in `loop3`. Delay is provided by means of a nested loop. This time, it is placed in a short subroutine “DELAY”.

** It is not necessary for PORTA pins since they are automatically configured as inputs after every reset.*

Source Code

```

1 ;*****
2 ;
3 ; Header
4 ;*****
5 ;***** DEFINING VARIABLES *****
6
7     cblock      0x20      ; Block of variables starts at address 20h
8     counter1    ; Variable "counter1" at address 20h
9     counter2    ; Variable "counter2" at address 21h
10    endc          ; Block of variables ends
11 ;***** MAIN PROGRAM *****
12
13     org      0x0000      ; Address of the first program instruction
14     banksel  ANSEL      ; Selects bank containing register ANSEL
15     clrf    ANSEL      ; Clears registers ANSEL and ANSELH to
16     clrf    ANSELH     ; configure all inputs as digital
17
18     banksel  TRISB      ; Selects bank containing register TRISB
19     clrf    TRISB      ; All port B pins are configured as outputs
20     movlw   B'00000010'
21     movwf   TRISA      ; Pin RA1 is input
22
23     banksel  PORTB      ; Selects bank containing register TRISB
24     movlw   B'00000001' ; Writes 1 to register W
25     movwf   PORTB      ; Number is moved to PORTB
26
27 loop   rlf     PORTB      ; Port B bits rotates by one place left
28       call    DELAY      ; Calls subroutine "DELAY"
29
30 loop3  btfss   PORTA,1    ; Tests the first port A bit
31       goto    loop3      ; "0" is applied to pin. Go to label "loop3"
32       goto    loop      ; "1" is applied to pin. Go to label "loop"
33
34 ;***** SUBROUTINES *****
35 DELAY
36     clrf     counter2    ; Clears variable "counter2"
37 loop1      clrf     counter1 ; Clears variable "counter1"
38
39 loop2      decfsz   counter1 ; Decrements variable "counter1" by 1
40           goto     loop2    ; Result is not 0. Go to label loop2
41           decfsz   counter2 ; Decrements variable "counter2" by 1
42           goto     loop2    ; Result is not 0. Go to label loop2
43           goto     loop1    ; Result is not 0. Go to label loop1
44       return      ; Return from subroutine "DELAY"
45
46     end          ; End of program

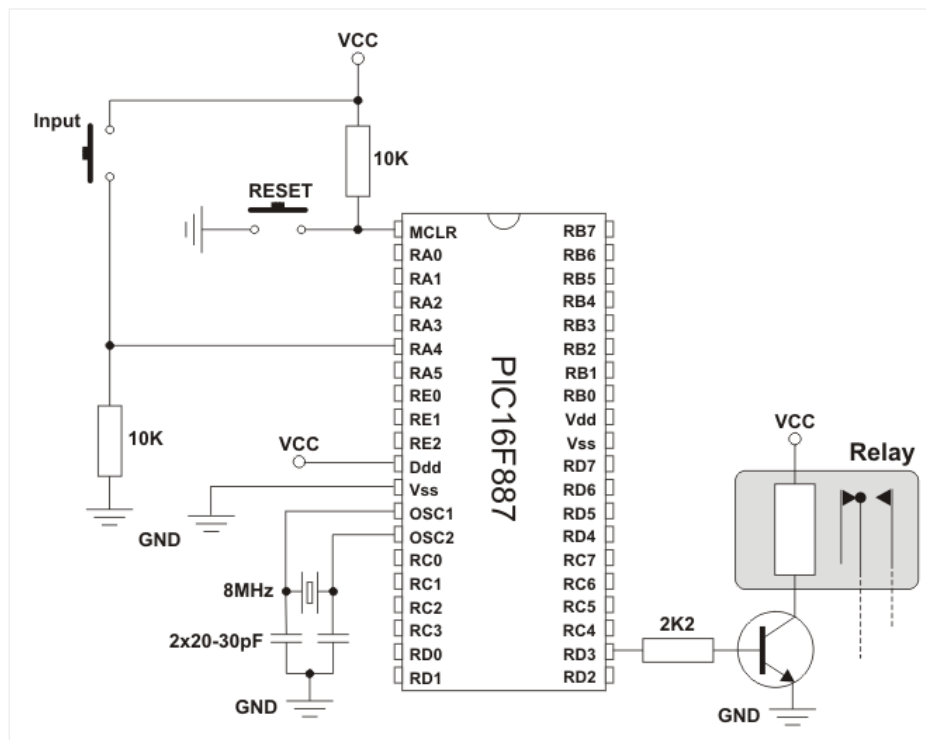
```

EXAMPLE 6

TMR0 as a counter, defining new variables, using relay

This time, TMR0 is used as a counter. The idea is to connect the counter input to one pushbutton so that it counts one pulse at a time upon every button press. When the number of counted pulses becomes equal to the number in register TEST, logic one voltage (5V) will be applied to the PORTD, 3 pin. Since this voltage activates an electro-mechanical relay, this bit is called the same- “Relay”.

In this example, the TEST register contains number 5. Naturally, it could be any number and could be calculated or entered via the keyboard. Instead of a relay, the microcontroller can activate some other device and instead of push-buttons it can use sensors. This example illustrates one of the most common uses of the microcontroller in industry. When something is done as many times as needed, then something else should be switched on or off...



Source Code

```

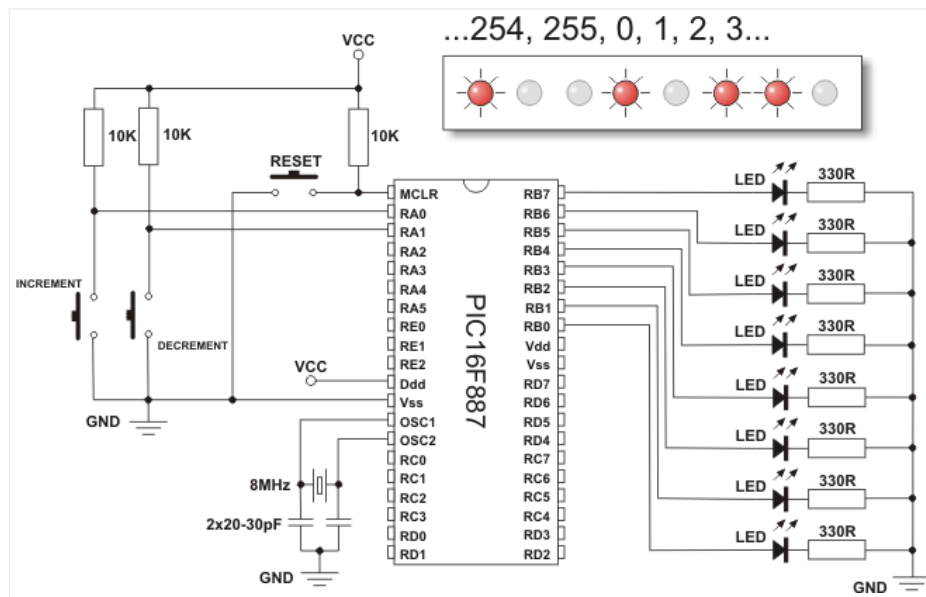
1  ;*****
2  ; Header
3  ;*****
4  ;***** DEFINING VARIABLES *****
5  ;*****
6      TEST      equ B'0000101' ; Binary number 0000101 = TEST
7      #define   RELAY PORTD,3 ; Pin PORTD,3 = RELAY
8  ;*****
9  ;***** MAIN PROGRAM *****
10 ;*****
11      org      0x0000 ; Address of the first program instruction
12
13      banksel   TRISB ; Selects bank containing register TRISB
14      clrf     TRISB ; All port B pins are configured as outputs
15      clrf     TRISD ; All port D pins are configured as outputs
16      movlw    B'00010000' ; This number is written to W register
17      movwf    TRISA ; Only the forth pin of port A is input
18
19      banksel   OPTION_REG ; Bank containing OPTION_REG register
20      bsf      OPTION_REG,T0CS ; Pin RA4 is supplied with pulses
21      bsf      OPTION_REG,PSA ; Prescaler rate is 1:1
22
23      banksel   PORTB ; Selects bank containing PORTB register
24
25      clrf     TMR0 ; Clears timer register
26      bcf      PORTD,3 ; Pin PORTD,3 = 0
27 loop
28      movfw    TMR0 ; Timer register is moved to W register
29      movwf    PORTB ; W register is moved to PORTB
30      xorlw    TEST ; Operation exclusive OR between
31                  ; W register and number TEST (0000101)
32      btfsc    STATUS,Z ; If numbers are equal, result is 0 and
33      bsf      PORTD,3 ; bit STATUS,Z = 1. Bit PORTD,3 is set
34      goto     loop ; and jump to label loop is executed
35
36      end ; End of program

```

EXAMPLE 7

Using macros in the program, using debounce routine

You have probably noticed in the previous example that the microcontroller does not always operate as expected. Namely, by pressing the push-button, the number on port B is not always incremented by 1. Mechanical push-buttons make several short successive contacts when they have been activated. You guess, the microcontroller registers and counts all that...



There are several ways to solve this problem. This program uses program delay known as *debounce*. Basically, it is a simple procedure. Upon input change detection (button press), a short program delay is provided and the program waits for another change (button release). Only after this, the program comes to a conclusion that the button is activated.

In this very case, the push-button is tested by means of macro called *button*. Besides, this macro contains a program delay which is provided by means of another macro *pausems*.

The main program is relatively simple and enables the variable “cnt” to be incremented and decremented by using two push-buttons. This variable is thereafter copied to port B and affects the LED (logic one (1) turns LED diode on, while logic zero (0) turns LED diode off).

Source Code

```

1 ;***** Header *****
2 ;***** DEFINING VARIABLES *****
3
4     cblock      0x20      ; Block of variables starts at address 20hex
5
6     HIcnt
7     LOcnt
8     LOOPcnt
9     cnt
10    endc
11 ;***** ; End of block of variables *****
12    ORG      0x000      ; Reset vector
13    nop
14    goto     main      ; Go to program start (label "main")
15 ;*****
16
17    include  "pause.inc"
18    include  "button.inc"
19
20 ;*****
21 main
22     banksel  ANSEL      ; Selects bank containing ANSEL
23     clrf     ANSEL      ; All pins are digital
24     clrf     ANSELH
25
26     banksel  TRISB
27     bsf      TRISA, 0
28     bsf      TRISA, 1
29     clrf     TRISB
30
31     banksel  PORTB
32     clrf     cnt
33
34 Loop
35     button   PORT,0,0,Increment
36     button   PORT,1,0,Decrement
37     goto     Loop
38
39 Increment
40     incf     cnt, f
41     movf     cnt, w
42     movwf    PORTB
43     goto     Loop
44
45 Decrement
46     decf     cnt, f
47     movf     cnt, w

```

```

46      movwf    PORTB
47      goto     Loop
48      end                      ; End of program

```

Macro “pausems”

```

1 ;*****
2 pausems MACRO arg1
3     local      Loop1
4     local      dechi
5     local      Delay1ms
6     local      Loop2
7     local      End
8
9     movlw      High(arg1)    ; Higher byte of argument is moved
10                                ; to H1cnt
11     movwf      H1cnt
12     movlw      Low(arg1)     ; Lower byte of argument is moved
13                                ; to L0cnt
14     movwf      L0cnt
15 Loop1
16     movf        L0cnt, f      ; Decrements H1cnt and L0cnt while
17     btfsc       STATUS, Z    ; needed and calls subroutine Delay1ms
18
19     goto        dechi
20     call        Delay1ms
21     decf        L0cnt, f
22     goto        Loop1
23 dechi
24     movf        H1cnt, f
25     btfsc       STATUS, Z
26     goto        End
27     call        Delay1ms
28     decf        H1cnt, f
29     decf        L0cnt, f
30     goto        Loop1
31 Delay1ms:
32     movlw       .100          ; Delay1ms provides delay of
33     movwf       L00Pcnt       ; 100*10us=1ms
34 Loop2:
35     nop
36     nop
37     nop
38     nop
39     nop
40     nop
41     nop
42     decfsz      L00Pcnt, f
43     goto        Loop2        ; Execution time of Loop2
44     return      ; is 10 us
45 End
46     ENDM
47 ;*****

```

Macro “button”

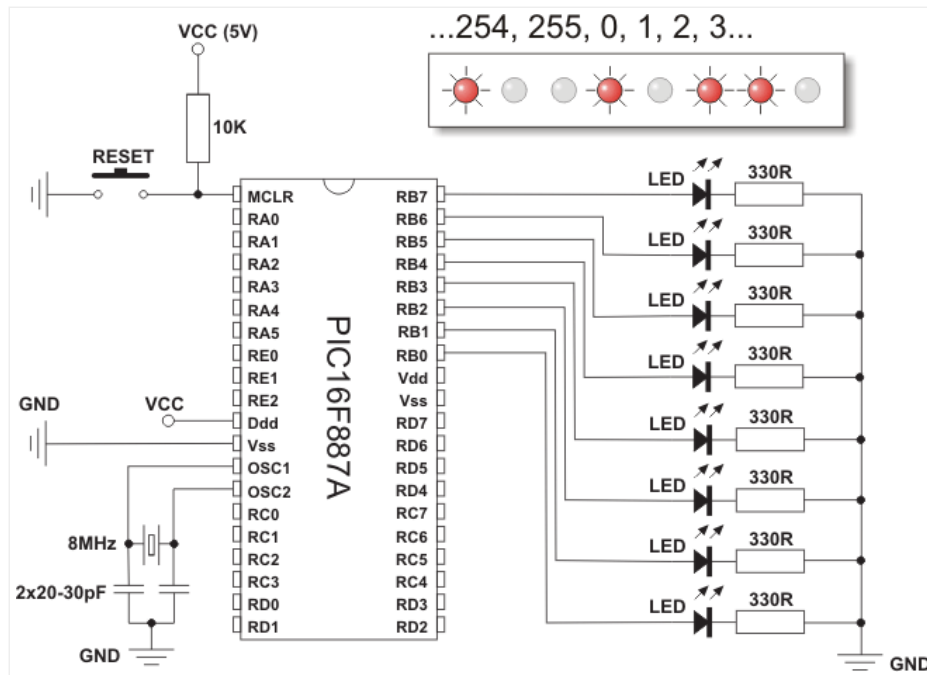
```

1 ;*****
2 button MACRO port,pin,hilo,label
3     local      Pressed1      ; All labels are local
4     local      Pressed2
5     local      Exit1
6     local      Exit2
7
8     IFNDEF     debouncedelay ; Enables debounce time to be defined
9                                ; in main program
10    #define     debouncedelay .10
11    ENDIF
12
13    IF (hilo == 0)             ; If pull-up used
14    btfsc       port, pin      ; If "1", push-button is pressed
15    goto        Exit1
16    pausems    debouncedelay ; Wait for 10ms debounce
17 Pressed1
18    btfss       port, pin
19    goto        Pressed1
20    pausems    debouncedelay ; Wait until released and
21    goto        label         ; jump to specified address
22 Exit1
23    ELSE
24    btfss       port, pin      ; If pull-down used
25    goto        Exit2         ; If "0", push-button is released
26    pausems    debouncedelay ; Wait for 10ms debounce
27 Pressed2
28    btfsc       port, pin
29    goto        Pressed2
30    pausems    debouncedelay ; Wait until released and
31    goto        label         ; jump to specified address
32 Exit2
33    ENDIF
34
35    ENDM
36 ;*****

```

EXAMPLE 8**Using timer TMR1 and using interrupt**

16-bit timer TMR1 is used in this example. By occupying its registers TMR1L and TMR1H, an interrupt occurs and the number on port B is incremented. This has already been seen in the previous examples. The difference is in the program delay which is a bit longer this time because the prescaler rate is 1:8.

**Source Code**

```

1  ;***** Header *****
2  ;***** DEFINING VARIABLES *****
3
4      cblock      0x20          ; Block of variables starts at address 20h
5      w_temp      ; Variable at address 20h
6      pclath_temp ; Variable at address 21h
7      status_temp ; Variable at address 22h
8      endc
9
10 ;***** PROGRAM START *****
11      org      0x0000          ; Address of the first program instruction
12      goto     main            ; Jump to label "main"
13
14 ;***** INTERRUPT ROUTINE *****
15
16      org      0x0004          ; Interrupt vector
17      movwf    w_temp          ; Save register W
18
19      movf     STATUS          ; Save register STATUS
20      movwf    status_temp
21
22      movf     PCLATH          ; Save register PCLATH
23      movwf    pclath_temp
24
25      banksel  PORTB           ; Selects bank containing PORTB
26      incf     PORTB           ; Register PORTB is incremented by 1
27
28      movf     pclath_temp,w    ; PCLATH is given its original content
29      movwf    PCLATH
30      movf     status_temp,w    ; STATUS is given its original content
31      movwf    STATUS
32      swapf    w_temp,f         ; W is given its original content
33      swapf    w_temp,w
34
35      banksel  PIR1            ; Selects bank containing PIR1
36      bcf      PIR1,TMR1IF      ; Clears interrupt flag TMR1IF
37
38      bsf      INTCON,GIE       ; Global interrupt enabled
39      retfie   ; Return from interrupt routine
40
41 ;***** MAIN PROGRAM *****
42
43 main
44      banksel  ANSEL           ; Start of main program
45      clrf     ANSEL           ; Selects bank containing register ANSEL
46      clrf     ANSELH          ; Clears registers ANSEL and ANSELH
47      clrf     ANSELH          ; All pins are digital

```

```

48     banksel    TRISB           ; Selects bank containing register TRISB
49     clrf       TRISB           ; All port B pins are configured as outputs
50
51     banksel    T1CON            ; Selects bank containing register T1CON
52     bcf        T1CON,TMR1CS     ; TMR1 counts pulses generated by oscillator
53
54     bsf        T1CON,T1CKPS0    ; Prescaler rate is 1:8
55     bsf        T1CON,T1CKPS1
56     bsf        T1CON,TMR1ON     ; Turns on timer TMR1
57
58     banksel    PIE1             ; Selects bank containing register PIE1
59     bsf        PIE1,TMR1IE      ; TMR1 interrupt overflow enabled
60     bsf        INTCON,PEIE      ; Peripheral modules interrupt enabled
61                                     ; Timer TMR1 belongs to peripheral modules
62     bsf        INTCON,GIE       ; Global interrupt enabled
63
64     banksel    PORTB            ; Selects bank containing register PORTB
65     clrf       PORTB           ; Clears port B
66 loop
67     goto       loop            ; Remain here
68     end

```

EXAMPLE 9

Using timer TMR2, configuring quartz oscillator

This example illustrates the use of timer TMR2. The microcontroller uses internal oscillator HFINTOSC with the frequency of 500 kHz. The whole program works as follows: After the period of time defined by register PR, prescaler and postscaler has expired, an interrupt occurs. Interrupt routine decrements the content of the PR register and simultaneously increments the content of port B. Since the number in register PR, which determines when interrupt is to occur is constantly decremented, interrupt will occur for shorter and shorter periods of time. In other words, counting will be carried out faster. A new cycle of accelerated counting starts after every register PR overflow.

Source Code

```

1  ;***** Header *****
2  ;***** DEFINING VARIABLES *****
3
4      cblock      0x20           ; Block of variables starts at address 20h
5      w_temp      ; Variable at address 20h
6      pclath_temp ; Variable at address 21h
7      status_temp ; Variable at address 22h
8      endc
9
10 ;***** PROGRAM START *****
11     org      0x0000           ; Address of the first program instruction
12     goto     main             ; Jump to label "main"
13
14 ;***** INTERRUPT ROUTINE *****
15
16     org      0x0004           ; Interrupt vector
17     movwf    w_temp           ; Save register W
18
19     movf     STATUS           ; Save register STATUS
20     movwf    status_temp
21
22     movf     PCLATH           ; Save register PCLATH
23     movwf    pclath_temp
24
25     banksel  PORTB            ; Selects bank containing PORTB
26     incf     PORTB            ; Increments PORTB register by 1
27     banksel  PR2              ; Selects bank containing PR2
28     decf     PR2              ; PR2 is decremented by 1
29     movf     pclath_temp,w    ; PCLATH is given its original state
30     movwf    PCLATH
31     movf     status_temp,w    ; STATUS is given its original state
32     movwf    STATUS
33     swapf    w_temp,f         ; W is given its original state
34     swapf    w_temp,w
35
36     banksel  PIR1             ; Selects bank containing PIR1
37     bcf      PIR1,TMR2IF      ; Clears interrupt flag TMR2IF
38
39     bsf      INTCON,GIE       ; Global interrupt enabled
40     retfie   ; Return from interrupt routine
41
42 ;***** MAIN PROGRAM *****
43 main    ; Start of the main program
44     banksel  OSCCON            ; Selects bank containing register OSCCON
45     bcf      OSCCON,6         ; Selects internal oscillator HFINTOSC with
46     bsf      OSCCON,5         ; frequency of 500KHz
47     bsf      OSCCON,4
48     bsf      OSCCON,0         ; Microcontroller uses internal oscillator
49
50     banksel  ANSEL            ; Selects bank containing register ANSEL

```

```

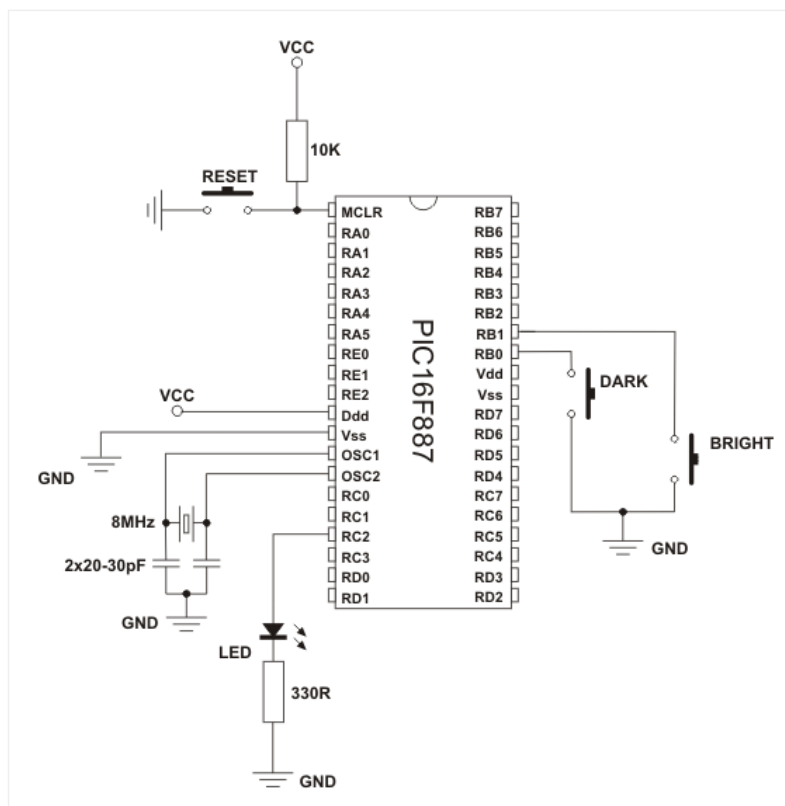
51      clrf      ANSEL      ; Clears registers ANSEL and ANSELH
52      clrf      ANSELH    ; All pins are digital
53
54      banksel    TRISB     ; Selects bank containing register TRISB
55      clrf      TRISB     ; All port B pins are configured as outputs
56      clrf      PR2
57
58      banksel    T2CON     ; Selects bank containing register T2CON
59      movlw     H'FF'      ; Sets all control register bits
60      movwf     T2CON     ; prescaler=1:16, postscaler=1:16 TMR2=ON
61      clrf      PORTB
62
63      banksel    PIE1      ; Selects bank containing register PIE1
64      bsf      PIE1,TMR2IE ; TMR2 interrupt enabled
65      bsf      NTCN,PEIE   ; Peripheral modules interrupt enabled
66
67      bsf      INTCON,GIE   ; Global interrupt enabled
68 loop
69      goto      loop       ; Remain here
70      end

```

EXAMPLE 10

Module CCP1 as PWM signal generator

Since the CCP modules have a wide range of possibilities they are commonly used in practice. This example illustrates the use of CCP1 module in PWM mode. Bits of the CCP1CON register determine that the module operates as a single-output PWM. The same bits determine the PWM frequency to be 4.88 kHz. To make things more interesting, the duration of the output P1A (PORTC,2) pulses may be changed by means of push-buttons symbolically called “DARK” and “BRIGHT”. Push-buttons are tested in interrupt routine initiated by the timer TMR1. Any change affects the LED diode so that it changes light intensity. Note that port B does not use external resistors because internal pull-up resistors are enabled. The whole process of generating PWM signal is performed “behind the scenes”, which enables the microcontroller to do other things.



Source Code

```

1 ;***** Header *****
2 ;***** DEFINING VARIABLES *****
3      cblock      0x20      ; Block of variables starts at address 20h
4      w_temp      ; Variable at address 20h
5      pclath_temp  ; Variable at address 21h
6      status_temp  ; Variable at address 22h
7      endc
8
9      #define      DARK PORTB,0 ; Push-button "DARK" is connected

```

```

10                                     ; to PORTB,0 pin
11      #define      BRIGHT PORTB,1  ; Push-button "BRIGHT" is connected
12                                     ; to PORTB,1 pin
13 ;***** PROGRAM START *****
14
15      org          0x0000            ; First program instruction address
16      goto         main             ; Jump to label "main"
17
18 ;***** INTERRUPT ROUTINE *****
19
20      org          0x0004            ; Interrupt vector
21
22      movwf        w_temp            ; Save register W
23
24      movf         STATUS            ; Save register STATUS
25      movwf        status_temp
26
27      movf         PCLATH            ; Save register PCLATH
28      movwf        pclath_temp
29
30      banksel      CCP1L             ;
31      btfss        DARK              ; Tests push-button "DARK"
32      decf         CCP1L             ; Push-button is pressed - decrement CCP1L by 1
33      btfss        BRIGHT            ; Testing push-button "BRIGHT"
34      incf         CCP1L             ; Push-button is pressed - increment CCP1L by 1
35
36      movf         pclath_temp,w     ; PCLATH is given its original content
37      movwf        PCLATH
38      movf         status_temp,w     ; STATUS is given its original content
39      movwf        STATUS
40      swapf        w_temp,f          ; W is given its original content
41      swapf        w_temp,w
42
43      banksel      PIR1              ; Selects bank containing PIR1
44      bcf          PIR1,TMR1IF       ; Clears interrupt flag TMR1IF
45
46      bsf          TMR1H,7           ; Accelerates timer TMR0 counting
47      bsf          TMR1H,6           ;
48      bsf          INTCON,GIE        ; Global interrupt enabled
49      retfie                          ; Return from interrupt routine
50
51 ;***** MAIN PROGRAM *****
52
53 main
54      banksel      ANSEL             ; Start of the main program
55      clrf         ANSEL             ; Selects bank containing register ANSEL
56      clrf         ANSELH           ; Clears registers ANSEL and ANSELH
57                                     ; All pins are digital
58
59      banksel      OPTION_REG        ; Selects bank containing register ANSEL
60      bcf          OPTION_REG,7      ; Pull-up resistors enabled
61      bsf          WPUB,0            ; Pull-up resistors enabled
62      bsf          WPUB,1            ; on port B pins 0 and 1
63
64      banksel      TRISC              ; Selects bank containing register TRISC
65      clrf         TRISC             ; All port C pins are configured as outputs
66
67      banksel      T1CON              ; Selects bank containing register T1CON
68      bcf          T1CON,TMR1CS       ; TMR1 operates as a timer
69      bcf          T1CON,T1CKPS0     ; Prescaler rate is 1:8
70      bsf          T1CON,TMR1ON      ; Activates timer TMR1
71
72      banksel      PIE1              ; Selects bank containing register PIE1
73      bsf          PIE1,TMR1IE       ; Interrupt TMR1 is enabled
74      bsf          INTCON,PEIE       ; Peripheral modules interrupts are
75                                     ; enabled
76      bsf          INTCON,GIE        ; Global interrupt enabled
77
78      movlw        B'1111101'        ; Prescaler TMR2 = 1:4
79      banksel      T2CON              ;
80      movwf        T2CON
81      movlw        B'1111111'        ; Number in register PR2
82      banksel      PR2
83      movwf        PR2
84
85      banksel      CCP1CON           ;
86      movlw        B'00001100'       ; Bits to configure CCP1 module
87      movwf        CCP1CON
88
89      loop
90      goto         loop             ; Remain here
91      end                          ; End of program

```

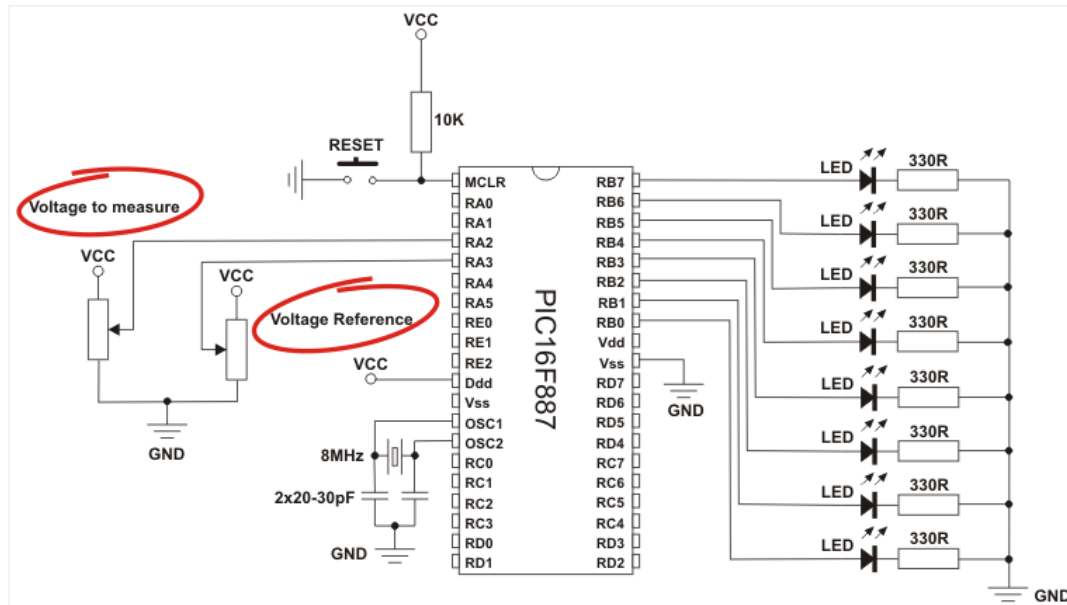
EXAMPLE 11

Using A/D converter

PIC16F887 A/D converter is used in this example. Everything is quite simple. A variable analog signal is applied on the AN2 pin while the result of conversion is shown on port B as a binary number. In order to simplify the program as much as possible, only 8 lower bits of the result of conversion are shown. GND is used as a negative voltage reference Vref-, while positive voltage reference is applied on the AN3 pin. It enables voltage

measurement scale to “stretch and shrink”.

To make this clear, the A/D converter always generates a 10-bit binary result, which means that it detects a total of 1024 voltage levels ($2^{10}=1024$). The difference between two voltage levels is not always the same. The less the difference between V_{ref+} and V_{ref} , the less the difference will be between two of 1024 levels. Accordingly, the A/D converter is able to detect slight changes in voltage.



Source Code

```

1  ;***** Header *****
2  ;***** PROGRAM START *****
3
4      org      0x0000      ; Address of the first program instruction
5
6      banksel  TRISB      ; Selects bank containing register TRISB
7      clrf    TRISB      ; All port B pins are configured as outputs
8      movlw   B'00001100'
9      movwf   TRISA      ; Pins RA2 and RA3 are configured as inputs
10
11     banksel  ANSEL      ; Selects bank containing register ANSEL
12     movlw   B'00001100'
13     movwf   ANSEL      ; Inputs AN2 and AN3 are analog while
14     clrf    ANSELH     ; all other pins are digital
15
16     banksel  ADCON1     ; Selects bank including register ADCON1
17     bsf     ADCON1,ADFM ; Right justification of result
18     bcf     ADCON1,VCFG1 ; Voltage Vss is used as Vref
19     bsf     ADCON1,VCFG0 ; RA3 pin voltage is used as Vref+
20
21     banksel  ADCON0     ; Selects bank containing register ADCON0
22     movlw   B'00001001' ; AD converter uses clock Fosc/2, AD channel
23     movwf   ADCON0     ; on RA2 pin is used for conversion and
24                     ; AD converter is enabled
25 loop
26     banksel  ADCON0     ; Tests bit GO/DONE
27     btfsc   ADCON0,1    ; Conversion in progress, remain in
28     goto    loop       ; loop
29
30     banksel  ADRESL     ; Lower byte of conversion result is
31     movf    ADRESL,w    ; copied to W
32
33     banksel  PORTB      ; Byte is copied to PORTB
34     movwf   PORTB      ; Starts new conversion
35     bsf     ADCON0,1    ; Jump to label "loop"
36     goto    loop
37     end              ; End of program

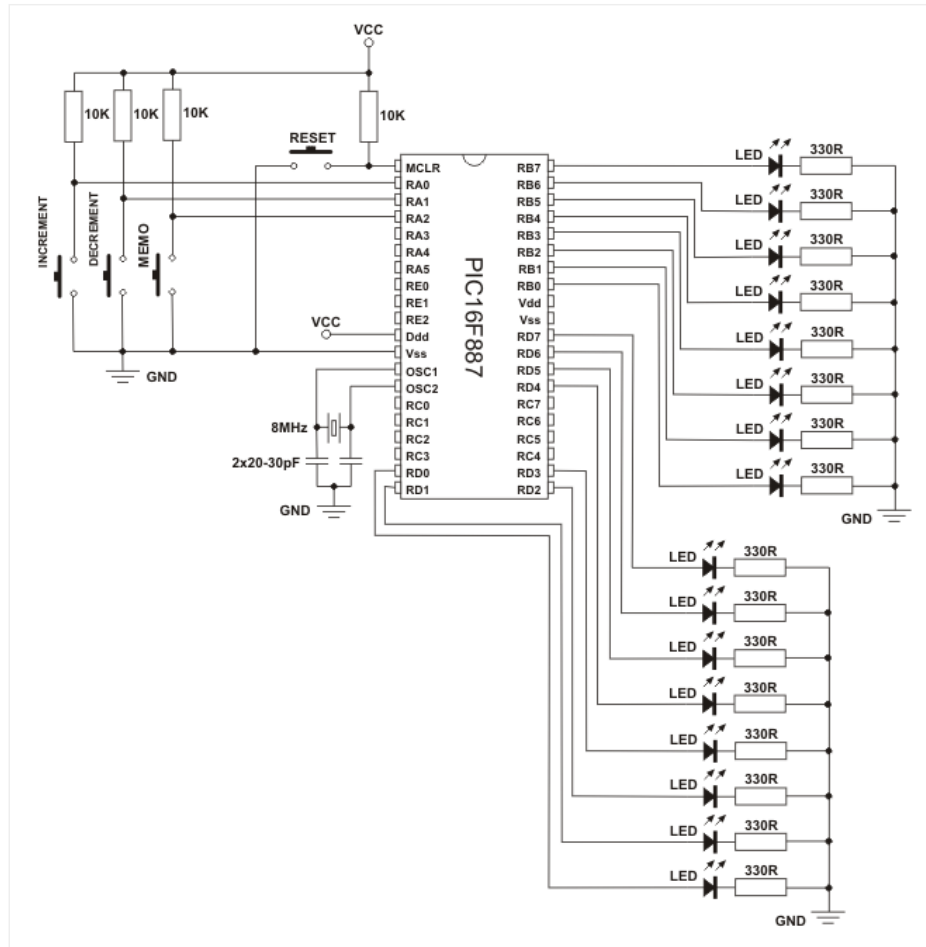
```

EXAMPLE 12

Using EEPROM memory

This example demonstrates write to and read from built-in EEPROM memory. The program works as follows. The main loop constantly reads EEPROM memory location at address 5 (decimal). This number is displayed on port D. The same loop tests the state of three push-buttons connected to port A. The push-buttons “INCREMENT” and

“DECREMENT” have the same purpose like in example 7 – increment and decrement the variable “cnt” which is thereafter displayed on port B. The push-button “MEMO” enables that variable to be written to EEPROM memory. In order to check it, it is enough to press this push-button and switch off the device. On the next switch on, the program displays the value of the variable on port D (at the moment of writing, this value was displayed on port B).



Source Code

```

1 ;***** Header *****
2 ;***** Defining variables in program *****
3     cblock    0x20        ; Block of variables starts at address 20h
4
5     HIcnt
6     LOcnt
7     LOOPcnt
8     cnt
9     endc                ; End of block
10
11 ;*****
12     ORG        0x000        ; Reset vector
13     nop
14     goto      main        ; Go to start of the program (label "main")
15 ;*****
16     include    "pause.inc"
17     include    "button.inc"
18 ;*****
19 main
20     banksel    ANSEL        ; Selects bank containing ANSEL
21     clrf      ANSEL
22     clrf      ANSELH        ; All pins are digital
23
24     banksel    TRISB
25     bsf       TRISA, 0        ; Input pin
26     bsf       TRISA, 1        ; Input pin
27     bsf       TRISA, 2        ; Input pin
28     clrf      TRISB        ; All port B pins are outputs
29     clrf      TRISD        ; All port D pins are outputs
30     banksel    PORTB
31     clrf      PORTB        ; PORTB=0
32     clrf      PORTD        ; PORTD=0
33     clrf      cnt          ; cnt=0
34 Loop
35     banksel    PORTA
36     button    PORTA,0,0,Increment
37     button    PORTA,1,0,Decrement

```

```

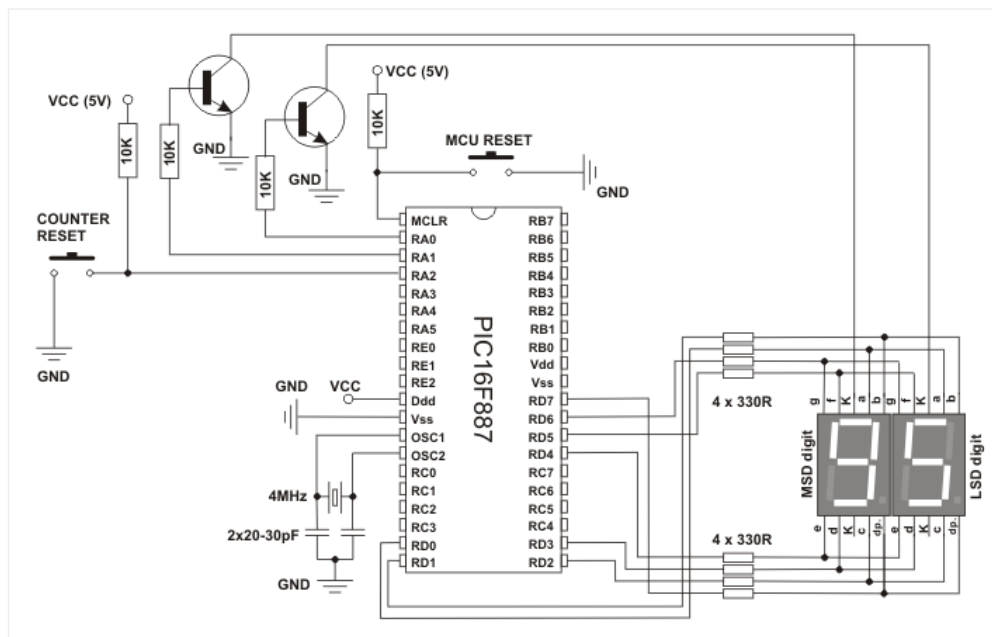
38      button    PORTA,2,0,Save
39
40      banksel   EEADR
41      movlw     .5           ; Reads EEPROM memory location
42      movwf     EEADR       ; at address 5
43      banksel   EECON1
44      bcf       EECON1,EEPGD
45      bsf       EECON1,RD   ; Reads data from EEPROM memory
46      banksel   EEDATA
47      movfw     EEDATA      ; Moves data to W
48      banksel   PORTD
49      movwf     PORTD      ; Data is moved from W to PORTD
50      goto      Loop
51 Increment    ; Increments number on port B
52      incf      cnt, f
53      movf      cnt, w
54      movwf     PORTB
55      goto      Loop
56 Decrement    ; Decrements number on port B
57      decf      cnt, f
58      movf      cnt, w
59      movwf     PORTB
60      goto      Loop
61 Save         ; Copies data from port B to EEPROM
62      banksel   EEADR      ; memory location at address 5
63      movlw     .5
64      movwf     EEADR      ; Writes address
65      banksel   PORTB
66      movfw     PORTB      ; Copies port B to register W
67      banksel   EEDAT
68      movwf     EEDAT      ; Writes data to temporary register
69      banksel   EECON1
70      bcf       EECON1,EEPGD
71      bsf       EECON1,WREN ; Write enabled
72
73      bcf       INTCON,GIE  ; All interrupts disabled
74      btfsc     INTCON,GIE
75      goto      $-2
76
77      movlw     55h
78      movwf     EECON2
79      movlw     H'AA'
80      movwf     EECON2
81      bsf       EECON1,WR
82
83      btfsc     EECON1,WR   ; Wait for write to complete
84      goto      $-1
85
86      bsf       INTCON,GIE  ; Interrupt enabled
87      bcf       EECON1,WREN
88      goto      Loop      ; Tests push-buttons again
89      end              ; End of program

```

EXAMPLE 13

Two-digit LED counter, multiplexing

In this example, the microcontroller operates as a two-digit counter. Concretely, the variable Dval is decremented (slow enough to be visible) and its value is displayed on twodigit LED display (99-0). The challenge is to enable binary number to be converted in decimal one and split it in two digits (tens and ones). Besides, since the LED display segments are connected in parallel, it is necessary to ensure that they change fast in order to make impression of simultaneous light emission (time-division multiplexing). Remember that in electronics, multiplexing allows several analog signals to be processed by one analog-to-digital converter (ADC). In this very case, time-division multiplexing is performed by the timer TMR0, while binary to decimal number conversion is performed in macro “digbyte”. Counter may be reset to its starting value (99) at any moment by pressing the pushbutton “COUNTER RESET”.



Source Code

```

1  ;***** Header *****
2  ;*****
3  ;
4      DEFINING VARIABLES IN PROGRAM
5      w_temp      EQU 0x7D      ; Variable for saving W register
6      status_temp EQU 0x7E      ; Variable for saving STATUS register
7      pclath_temp EQU 0x7F      ; Variable for saving PCLATH register
8
9      CBLOCK      0x20          ; Block of variables starts at address 20h
10     Digtemp
11
12     Dig0          ; Variables for displaying digits - LSB
13     Dig1
14     Dig2
15     Dig3          ; Variables for displaying digits - MSB
16     Dval          ; Counter value
17     One           ; Auxiliary variable which determines which
18                   ; display is to be switched on
19
20
21     ENDC          ; End of block of variables
22
23     poc_vr       EQU .99      ; Initial counter value is 99
24
25     include      "Digbyte.inc"
26
27 ;*****
28 ;*****
29     ORG          0x0000      ; First instruction address
30     goto         main       ; Jump to label "main"
31 ;*****
32 ;*****
33     ORG          0x0004      ; Interrupt vector address
34
35     movwf        w_temp      ; Move w register to w_temp register
36
37
38     movf         STATUS,w    ; Move STATUS register to status_temp
39     movwf        status_temp ; register
40
41     movf         PCLATH,w    ; Move PCLATH register to pclath_temp
42     movwf        pclath_temp ; register
43
44     ; Start of interrupt routine...
45
46     BANKSEL      TMR0
47     movlw        .100
48     movwf        TMR0
49     bcf          INTCON, T0IF
50
51     bcf          PORTA, 0
52     bcf          PORTA, 1
53     btfsc       One, 0
54     goto        Lsdon
55     goto        Msdon
56
57 Lsdon
58     incf         One, f
59     movlw        HIGH (Bcdto7seg)
60     movwf        PCLATH
61     digbyte      Dval
62     movf         Dig1, w
63     call         Bcdto7seg    ; Place L1 mask on the PORTD
64     movwf        PORTD
65     bsf          PORTA, 1

```

```

64      goto      ISR_end
65
66 Msdon
67      incf       One, f
68      movlw     HIGH (Bcdto7seg)
69      movwf     PCLATH
70      digbyte   Dval
71      movf      Dig0, w
72      call      Bcdto7seg      ; Place LO mask on the PORTD
73      movwf     PORTD
74      bsf       PORTA, 0
75      goto      ISR_end
76
77      ; End of interrupt routine...
78
79 ISR_end
80      movf      pclath_temp,w  ; PCLATH register is given its original
81      movwf     PCLATH        ; state
82
83      movf      status_temp,w ; STATUS register is given its original
84      movwf     STATUS        ; state
85
86      swapf     w_temp,f      ; W register is given its original
87                                ; state
88      swapf     w_temp,w
89      retfie     ; Return from interrupt routine
90
91 main
92      banksel   ANSEL        ; Selects bank containing ANSEL
93      clrf      ANSEL        ; All pins are digital
94      clrf      ANSELH
95
96      BANKSEL   TRISA
97      movlw     b'11111100'   ; RA0 and RA1 are configured as outputs and
98                                ; used for 7-segment display multiplexing
99                                ; RA2 is input push-button for initializa
100                                ; tion
101
102      movwf     TRISA
103      clrf      TRISD
104
105      BANKSEL   OPTION_REG
106      movlw     b'10000110'   ; TMR0 is incremented each 32us (Fclk=8MHz)
107      movwf     OPTION_REG
108
109      BANKSEL   PORTA
110      movlw     poc_vr
111      movwf     Dval          ; Dval contains counter value
112      movlw     b'00000001'   ; Initializes variable specifying display
113      movwf     One           ; to switch on
114      movwf     PORTA
115      movlw     .100
116      movwf     TMR0          ; TMR0 interrupt appr.every 10ms
117      bsf       INTCON, GIE    ; Global interrupt enabled
118      bsf       INTCON, T0IE   ; Timer TMR0 interrupt enabled
119      bcf       INTCON, T0IF
120
121 Loop
122      btfss     One, 3          ; Falling edge encountered?
123      goto      Dec            ; Yes! Go to Dec
124      btfss     PORTA, 2        ; Counter reset button pressed?
125      goto      Reset          ; Yes! Go to Reset
126      ; Decrement Dval counter by 1
127      Dec
128      btfss     One, 3
129      goto      Dec
130      movf      Dval, f
131      btfsc     STATUS, Z       ; Is Dval equal to 0?
132      goto      Loop           ; If it is, go to loop and wait for T2
133      decf      Dval, f         ; If Dval not equal to 0, decrement it by 1
134      goto      Loop
135
136 Reset
137      btfss     PORTA, 2        ; Wait for rising edge
138      goto      Reset
139      movlw     poc_vr
140      movwf     Dval           ; Write initial value to counter
141      goto      Loop
142
143 ;*****
144 ORG      0x0300               ; Lookup table is at the top of third page, but
145                                ; can be placed at some other place, it is impor
146                                ; tant to have it all on one page
147
148 Bcdto7seg
149      addwf     PCL, f
150      DT        0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f
151 ;*****
152      END                ; End of program

```

Macro “digbyte”:

```

1 digbyte MACRO arg0
2     LOCAL    Exit0
3     LOCAL    Exit1
4     LOCAL    Exit2
5
6     clrf     Dig0
7     clrf     Dig1
8     clrf     Dig2

```

```

9      clrf      Dig3
10
11     movf      arg0, w
12     movwf     Digtemp
13     movlw     .100
14 Exit2  incf      Dig2, f
15         subwf     Digtemp, f
16         btfsc     STATUS, C
17         goto      Exit2
18     decf      Dig2, f
19     addwf     Digtemp, f
20
21 Exit1  movlw     .10
22         incf      Dig1, f
23         subwf     Digtemp, f
24         btfsc     STATUS, C
25         goto      Exit1
26     decf      Dig1, f
27     addwf     Digtemp, f
28
29 Exit0  movf      Digtemp, w
30         movwf     Dig0
31     ENDM
32

```

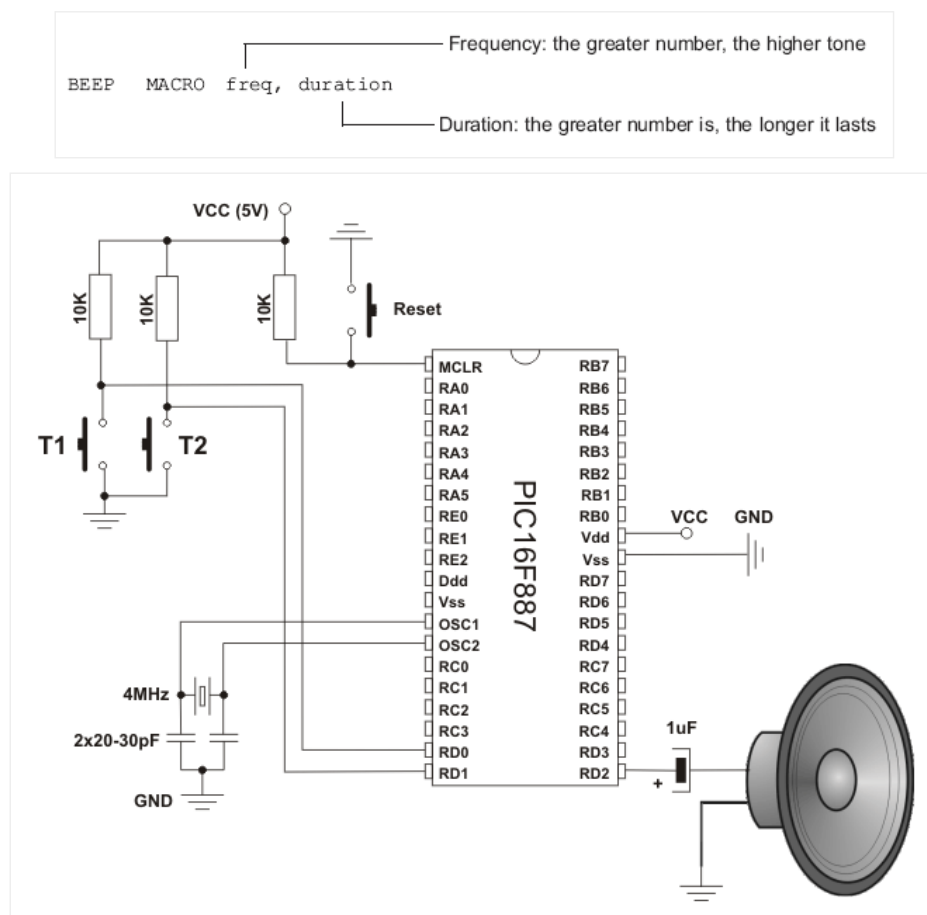
Macro digbyte is used to convert the number from digital to decimal format. Besides, digits of such decimal number are stored into special registers in order to enable them to be displayed on LED displays.

EXAMPLE 14

Sound generating, using macros

The generation of sound is a task commonly assigned to the microcontroller. Basically, it all comes to generating a pulse sequence on one output pin. While doing so, the proportion of logic zero (0) to logic one (1) duration determines the tone pitch and by changing different tones, different melodies arise.

In this example, any press on push-buttons T1 and T2 generates a sound. The appropriate instructions are stored in macro “beep” containing two arguments.



Source Code

```

1 ;***** Header *****
2 ;***** Defining variables in program *****
3     cblock      0x20
4     Hicnt
5     LOcnt
6     LOOPcnt
7     PRESCwait
8     Beep_TEMP1
9     Beep_TEMP2
10    Beep_TEMP3
11    endc
12
13 #define BEEPport PORTD, 2
14 #define BEEPtris TRISD, 2
15
16    expand
17
18 ;*****
19    ORG      0x0000
20    goto     main
21 ;*****
22 ; remaining code goes here
23
24    include   "pause.inc"
25    include   "button.inc"
26    include   "beep.inc"
27
28 main
29    banksel   ANSEL
30    clrf      ANSEL
31    clrf      ANSELH
32
33    banksel   TRISD
34    movlw     b'11111011'
35    movwf     TRISD
36    banksel   PORTD
37    BEEPinit
38
39    Loop
40    button    PORTD,0,0,Play1
41    button    PORTD,1,0,Play2
42    goto     Loop
43
44    Play1
45    BEEP      0xFF, 0x02
46    BEEP      0x90, 0x05
47    BEEP      0xC0, 0x03
48    BEEP      0xFF, 0x03
49    goto     Loop
50
51    Play2
52    BEEP      0xBB, 0x02
53    BEEP      0x87, 0x05
54    BEEP      0xA2, 0x03
55    BEEP      0x98, 0x03
56    goto     Loop
57
58 ;*****
59 END

```

Macro “beep”:

```

1 BEEPinit MACRO
2     bcf      STATUS, RP0
3     bcf      STATUS, RP1
4     bcf      BEEPport
5     bsf      STATUS, RP0
6     bcf      STATUS, RP1
7     bcf      BEEPtris
8     movlw     b'00000111'
9     movwf     OPTION_REG
10    bcf      STATUS, RP0
11    bcf      STATUS, RP1
12    ENDM
13 BEEP MACRO freq, duration
14     bcf      STATUS, RP0
15     bcf      STATUS, RP1
16     movlw     freq
17     movwf     Beep_TEMP1
18     movlw     duration
19     movwf     Beep_TEMP2
20     call      BEEPsub
21     ENDM
22 ;*****
23 ; Subroutines
24
25 BEEPsub
26     clrf      TMR0
27     bcf      INTCON, T0IF
28     bcf      BEEPport
29
30 BEEPa
31     bcf      INTCON, T0IF
32
33 BEEpb
34     bsf      BEEPport
35     call      B_Wait
36     bcf      BEEPport
37     call      B_Wait
38     btfss    INTCON, T0IF
39     goto     BEEpb

```

```

38     decfsz    Beep_TEMP2, f    ; Is Beep_TEMP2 = 0 ?
39     goto     BEEPa            ; Go to BEEPa again
40     return
41 B_Wait
42     movf      Beep_TEMP1, w
43     movwf     Beep_TEMP3
44 B_Waita
45     decfsz    Beep_TEMP3, f
46     goto     B_Waita
47     return

```

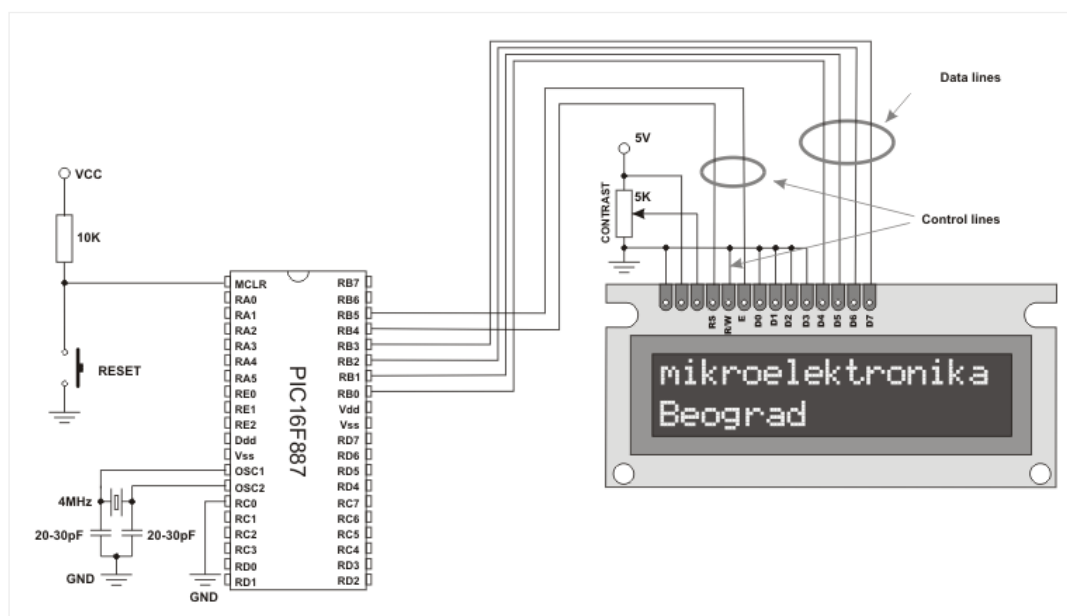
EXAMPLE 15

Using LCD display

This example illustrates the use of the alphanumeric LCD display. The program itself is very simple because macros are used (usually the effort of creating Macros pays off in the end).

Two messages written on two lines change on display. The second message is intended to display the current temperature. Since no sensor is installed, the measurement is not really carried out, the variable “temp” appears on the display instead of the measured temperature.

In reality, the current temperature or some other measured value would be displayed.



Source Code

```

1 ;***** Header *****
2 ; DEFINING VARIABLES IN PROGRAM
3
4 CBLOCK    0x20    ; Block of variables starts at address 20h
5
6 HICnt     ; Belongs to macro "pausems"
7 L0Cnt
8 LOOPCnt
9
10 LCDbuf    ; Belongs to functions "LCDxxx"
11 LCDtemp
12 LCDportBuf ; LCD Port Buffer
13
14 Digtemp   ; Belongs to macro "digbyte"
15 Dig0
16 Dig1
17 Dig2
18 Dig3
19
20 temp
21 ENDC      ; End of block
22
23 LCDport EQU PORTB ; LCD is on PORTB (4 data lines on RB0-RB3)
24 RS      EQU 4      ; RS line connected to RB4
25 EN      EQU 5      ; EN line connected to RB5
26
27 ;*****
28 ORG     0x0000    ; Reset vector address
29 nop
30 goto    main      ; Go to beginning of the program (label "main")

```

```

31 ;*****
32     include    "LCD.inc"
33     include    "digbyte.inc"
34     include    "pause.inc"
35 ;*****
36 main
37     bankssel    ANSEL            ; Selects bank containing ANSEL
38     clrf        ANSEL            ; All pins are digital
39     clrf        ANSELH
40
41     bcf         STATUS,RP0        ; Bank0 active only
42     bcf         STATUS,RP1
43     movlw       .23
44     movwf       temp              ; Move arbitrary value to variable
45                                     ; is to be displayed on LCD
46     lcdinit     ; LCD initialization
47 Loop
48     lcdcmd      0x01              ; Instruction to clear LCD
49     lcdtext     1, "mikroelektronika" ; Write text from the begin
50                                     ; ning of the first line
51     lcdtext     2, "Beograd"      ; Write text from the beginning of
52                                     ; the second line
53     pausems     .2000             ; 2 sec. delay
54     lcdcmd      0x01              ; Instruction to clear LCD
55     lcdtext     1, "Temperatura1" ; Write text from the begin
56                                     ; ning of the first line
57     lcdtext     2, "temp="         ; Write text from the beginning of
58                                     ; the second line
59     lcdbyte     temp              ; Write variable (dec.)
60     lcdtext     0, " C"           ; Write text after cursor
61     pausems     .2000             ; 2 sec. delay
62     goto        Loop
63 ;*****
64     end                    ; End of program

```

LCD.inc

```

1 ;*****
2 ; Initialization must be done by using macro lcdinit before access
3 ; ing LCD
4 ;*****
5 lcdinit MACRO
6     bcf         STATUS, RP0        ; Bank0
7     bcf         STATUS, RP1
8     clrf        LCDportBuf
9     movf        LCDportBuf, w
10    movwf       LCDport
11    bsf         STATUS, RP0        ; Bank1
12    bcf         STATUS, RP1
13    clrf        TRISB              ; LCDport with output LCD
14    bcf         STATUS, RP0        ; Bank0
15    bcf         STATUS, RP1
16
17 ; Function set (4-bit mode change)
18    movlw       b'00100000'
19    movwf       LCDbuf
20    swapf       LCDbuf, w
21    movwf       LCDportBuf
22    bcf         LCDportBuf, RS
23    movf        LCDportBuf, w
24    movwf       LCDport
25    bsf         LCDportBuf, EN
26    movf        LCDportBuf, w
27    movwf       LCDport
28    bcf         LCDportBuf, EN
29    movf        LCDportBuf, w
30    movwf       LCDport
31    call        Delay1ms            ; 1 ms delay
32
33 ; Function set (display mode set)
34    lcdcmd      b'00101100'
35    call        Delay1ms            ; 1 ms delay
36
37 ; Display ON/OFF Control
38    lcdcmd      b'00001100'
39    call        Delay1ms            ; 1 ms delay
40
41 ; Entry Mode Set
42    lcdcmd      b'00000110'
43    call        Delay1ms            ; 1 ms delay
44
45 ; Display Clear
46    lcdcmd      b'00000001'
47    pausems     .40                ; 40 ms delay
48
49 ; Function set (4-bit mode change)
50    movlw       b'00100000'
51    movwf       LCDbuf
52    swapf       LCDbuf, w
53    movwf       LCDportBuf
54    bcf         LCDportBuf, RS
55    movf        LCDportBuf, w
56    movwf       LCDport
57    bsf         LCDportBuf, EN
58    movf        LCDportBuf, w
59    movwf       LCDport

```

```

60      bcf      LCDportBuf, EN
61      movf     LCDportBuf, w
62      movwf    LCDport
63      call     Delay1ms      ; 1 ms delay
64
65 ; Function set (display mode set)
66      lcdcmd   b'00101100'
67      call     Delay1ms      ; 1 ms delay
68
69 ; Display ON/OFF Control
70      lcdcmd   b'00001100'
71      call     Delay1ms      ; 1 ms delay
72
73 ; Entry Mode Set
74      lcdcmd   b'00000110'
75      call     Delay1ms      ; 1 ms delay
76
77 ; Display Clear
78      lcdcmd   b'00000001'
79      pausems  .40           ; 40 ms delay
80
81      ENDM
82
83 ;*****
84 ; lcdcmd sends command to LCD (see the table on the previous page)
85 ; lcdclr is the same as lcdcmd 0x01
86 ;*****
87      MACRO    LCDcommand      ; Send command to LCD
88      movlw    LCDcommand
89      call     LCDcomd
90      ENDM
91
92      LCDcomd
93      movwf    LCDbuf
94      bcf      LCDportBuf, RS
95      movf     LCDportBuf, w
96      movwf    LCDport
97      goto     LCDwr
98
99      LCDdata
100     movwf    LCDbuf
101     bsf      LCDportBuf, RS
102     movf     LCDportBuf, w
103     movwf    LCDport
104     goto     LCDwr
105
106     LCDwr
107     swapf    LCDbuf, w
108     call     SendW
109     movf     LCDbuf, w
110     call     SendW
111     return
112
113     SendW
114     andlw    0x0F
115     movwf    LCDtemp
116
117     movlw    0xF0
118     andwf    LCDportBuf, f
119     movf     LCDtemp, w
120     iorwf    LCDportBuf, f
121     movf     LCDportBuf, w
122     movwf    LCDport
123     call     Delay1ms
124     bsf      LCDportBuf, EN
125     movf     LCDportBuf, w
126     movwf    LCDport
127     bcf      LCDportBuf, EN
128     call     Delay1ms
129     return
130 ;*****
131 ; lcdtext writes text containing 16 characters which represents a
132 ; macro argument. The first argument select selects the line in which
133 ; text writing is to start. If select is 0, text writing starts from
134 ; cursor current position.
135 ;*****
136      MACRO    select, text      ; This macro writes text from cursor
137                                  ; current position. Text is specified
138                                  ; in argument consisting of 16 charac
139                                  ; ters
140      local    Message
141      local    Start
142      local    Exit
143      local    i=0
144      goto     Start
145      Message  DT text          ; Create lookup table from arguments
146      DT       0
147
148      Start
149      IF (select == 1)
150      lcdcmd b'10000000'
151      ELSE
152      IF (select == 2)
153      lcdcmd b'11000000'
154      ENDF
155      ENDF
156
157      WHILE (i<16)              ; Repeat conditional program compiling 16 times
158      call Message+i            ; Read lookup table and place value in W
159      addlw 0

```

```

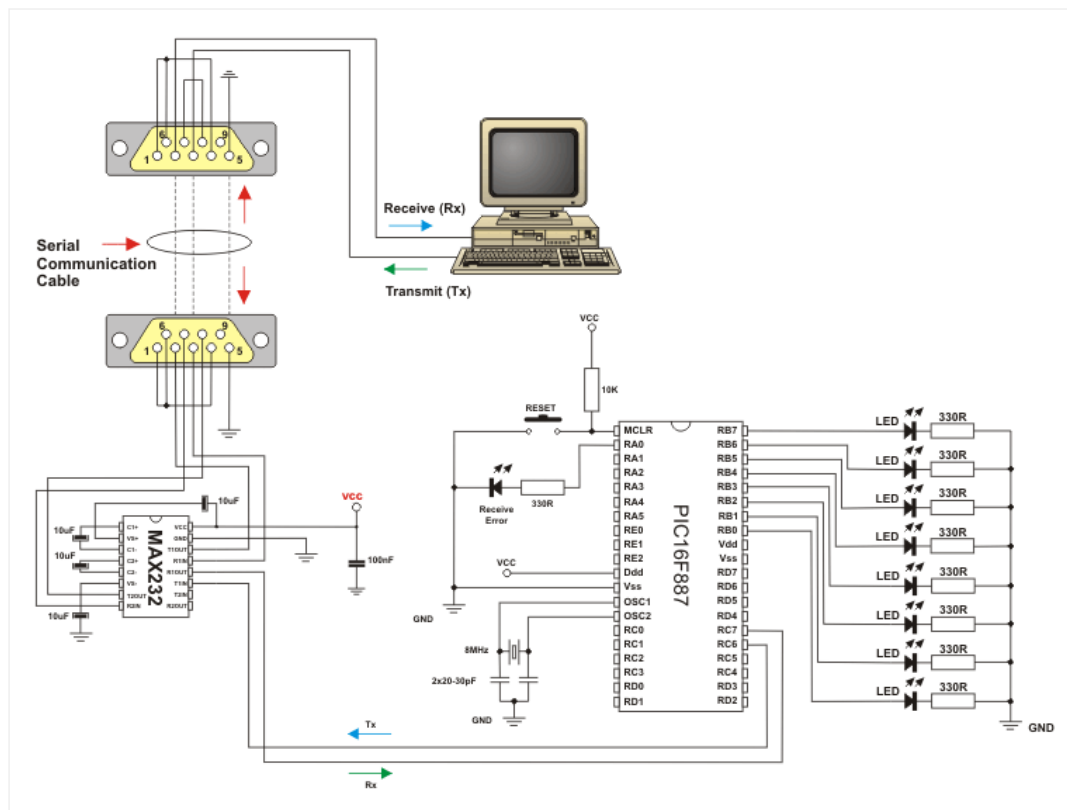
159      bz Exit                ; until 0 is read
160      call LCDdata           ; Call routine displaying W on LCD
161      i=i+1
162      ENDW
163 Exit
164      ENDM
165
166 ;*****
167 ; This macro writes value in size of 1 byte on LCD
168 ; excluding leading zeros
169 ;*****
170 lcdbyte MACRO arg0
171     digbyte    arg0          ; A hundred is in Dig2,
172                                ; A ten is in Dig1 and one in Dig0
173     movf       Dig2, w
174     addlw      0x30
175     call       LCDdata
176     movf       Dig1, w      ; If digit is 0 move cursor
177     addlw      0x30
178     call       LCDdata
179     movf       Dig0, w      ; If digit is 0 move cursor
180     addlw      0x30
181     call       LCDdata
182     ENDM
183 ;*****
184 ; 1ms Delay
185 Delay1ms:
186     movlw      .200
187     movwf      L00Pcnt
188 Delay10us:
189     nop                    ;1us
190     nop                    ;1us
191     nop                    ;1us
192     nop                    ;1us
193     nop                    ;1us
194     nop                    ;1us
195     nop                    ;1us
196     decfsz     L00Pcnt, f   ;1us
197     goto       Delay10us   ;2us
198
199     return

```

EXAMPLE 16

RS232 serial communication

This example illustrates the use of the microcontroller's EUSART module. Connection to the PC is enabled through RS232 standard. The program works in the following way: Every byte received via the serial communication is displayed using LED diodes connected to port B and is automatically returned to the transmitter thereafter. If an error occurs on receive, it will be signalled by switching the LED diode on. The easiest way to test device operation in practice is by using a standard Windows program called *Hyper Terminal*.



Source Code

```

1  ;***** Header *****
2  ;
3  ;      DEFINING VARIABLES IN PROGRAM
4
5      w_temp    EQU 0x7D    ; Variable for saving W register
6      status_temp EQU 0x7E    ; Variable for saving STATUS register
7      pclath_temp EQU 0x7F    ; Variable for saving PCLATH w register
8
9      cblock    0x20        ; Block of variables starts at address 20 h
10     Port_A      ; Variable at address 20 h
11     Port_B      ; Variable at address 21 h
12     RS232temp   ; Variable at address 22 h
13     RXchr       ; Variable at address 23 h
14     endc        ; End of block of variables
15 ;*****
16 ;
17 ;      ORG 0x0000 ; Reset vector
18 ;
19 ;      goto main ; Go to beginning of program (label "main")
20 ;*****
21 ;
22 ;      ORG 0x0004 ; Interrupt vector address
23 ;*****
24 ;
25 ;      movwf w_temp ; Save value of W register
26 ;      movf STATUS,w ; Save value of STATUS register
27 ;      movwf status_temp
28 ;      movf PCLATH,w ; Save value of PCLATH register
29 ;      movwf pclath_temp
30 ;*****
31 ;
32 ;      This part of the program is executed in interrupt routine
33 ;
34 ;      banksel PIE1
35 ;      btfss PIE1, RCIE
36 ;      goto ISR_Not_RX232int
37 ;      banksel PIE1
38 ;      btfsc PIR1, RCIF
39 ;      call RX232_int_proc
40 ;
41 ;
42 ;      ISR_Not_RX232int
43 ;
44 ;      movf pclath_temp,w
45 ;      movwf PCLATH ; PCLATH is given its original value
46 ;
47 ;      movf status_temp,w
48 ;      movwf STATUS ; STATUS is given its original value
49 ;      swapf w_temp,f
50 ;      swapf w_temp,w ; W is given its original value
51 ;
52 ;      retfie ; Return from interrupt routine
53 ;*****
54 ;
55 ;      RX232_int_proc ; Check if error has occurred
56 ;
57 ;      banksel RCSTA
58 ;      movf RCSTA, w
59 ;      movwf RS232temp
60 ;      btfsc RS232temp, FERR
61 ;      goto RX232_int_proc_FERR
62 ;      btfsc RS232temp, OERR
63 ;      goto RX232_int_proc_OERR
64 ;      goto RX232_int_proc_Cont

```

```

56 RX232_int_proc_FERR
57     bcf        RCSTA, CREN    ; To clear FERR bit, receiver is first
58                                     ; switched off and on afterwards
59     nop                                     ; Delay ...
60     nop
61     bsf        RCSTA, CREN
62     movf       RCREG, w         ; Reads receive register and clears FERR bit
63     bsf        Port_A, 0       ; Switches LED on ( UART error indicator)
64     movf       Port_A, w
65     movwf      PORTA
66     goto      RS232_exit
67
68 RX232_int_proc_OERR
69     bcf        RCSTA, CREN    ; Clears OERR bit
70     nop                                     ; Delay ...
71     nop
72     bsf        RCSTA, CREN
73     movf       RCREG, w         ; Reads receive register and clears FERR bit
74     bsf        Port_A, 1       ; Switches LED on ( UART error indicator)
75     movf       Port_A, w
76     movwf      PORTA
77     goto      RS232_exit
78
79 RX232_int_proc_Cont
80     movf       RCREG, W         ; Reads received data
81     movwf      RXchr
82     movwf      PORTB
83     movwf      TXREG           ; Sends data back to PC
84
85 RS232_exit
86     return      ; Return from interrupt routine
87 ;*****
88 ; Main program
89
90 main
91     banksel    ANSEL           ; Selects bank containing ANSEL
92     clrf       ANSEL           ; All inputs are digital
93     clrf       ANSELH
94
95     ;-----
96     ; Port configuration
97     ;-----
98     banksel    TRISA
99     movlw      b'11111100'
100    movwf      TRISA
101    movlw      b'00000000'
102    movwf      TRISB
103    ;-----
104    ; Setting initial values
105    ;-----
106    banksel    PORTA
107    movlw      b'11111100'
108    movwf      PORTA
109    movwf      Port_A
110    movlw      b'00000000'
111    movwf      PORTB
112    movwf      Port_B
113    ;-----
114    ; USART - setting for 38400 bps
115    ;-----
116    banksel    TRISC
117    bcf        TRISC, 6         ; RC6/TX/CK = output
118    bsf        TRISC, 7         ; RC7/RX/DT = input
119
120    banksel    BAUDCTL
121    bsf        BAUDCTL, BRG16
122    banksel    SPBRG
123    movlw      .51              ; baud rate = 38400
124                                     ; ( Fosc/(4*(SPBRG+1))) Error +0.16%
125    movwf      SPBRG
126    clrf       SPBRGH
127
128    banksel    TXSTA
129    bcf        TXSTA, TX9       ; Data is 8-bit wide
130    bsf        TXSTA, TXEN      ; Data transmission enabled
131    bcf        TXSTA, SYNC      ; Asynchronous mode
132    bsf        TXSTA, BRGH      ; High-speed Baud rate
133
134    banksel    RCSTA
135    bsf        RCSTA, SPEN      ; RX/DT and TX/CK outputs configuration
136    bcf        RCSTA, RX9       ; Select mode for 8-bit data receive
137    bsf        RCSTA, CREN      ; Receive data enabled
138    bcf        RCSTA, ADDEN     ; No address detection, ninth bit may be
139                                     ; used as parity bit
140    movf       RCSTA, W
141    movf       RCREG, W
142    ;-----
143    ; Interrupts enabled
144    ;-----
145    banksel    PIE1
146    bsf        PIE1, RCIE      ; USART Rx interrupt enabled
147
148    bsf        INTCON, PEIE     ; All peripheral interrupts enabled
149    bsf        INTCON, GIE      ; Global interrupt enabled
150
151    ;-----
152    ; Remain here
153    ;-----
154    goto      $

```

```
155  
156      end                ; End of program
```



Appendix B. Examples by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.